Requirements and Workflow

Epic 1: Authentication and Access Control

This epic provides the functionality for staff members to sign in to the fare configuration application with a username or email and a password. After login, each user sees a landing page tailored to their role (`OPERATOR` vs. `CLIENT_SERVICES`). Operators receive standard access and can change their own password. Caracal Administrators (`CLIENT_SERVICES`) can create accounts, modify roles, and view all registered users. Accounts are provisioned administratively with no public sign-up. Passwords are encrypted as BCrypt hashes, sessions are persisted in PostgreSQL, and all protected endpoints require authentication. Google Single Sign-On and the Caracal API identity header are slated for Iteration 2.

Story 1.1: Log In with Username or Email

- **GitHub Title:** `[Auth] Log In with Username or Email`
- **Iteration:** `1`
- **Story Points:** `2`
- **Story Description:** As a registered staff member, I want to log in with either my username or my email plus my password, so that I can securely access the system.
- **Actors / Personas:** Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**
  - *Precondition:* An account exists in the `app_users` table and the user is currently signed out.
  - *Trigger:* The user navigates to `/login` and submits the authentication form.
- **Actions / Postconditions:**
  - *Action:* The system looks up the account by username or email and verifies the submitted password against the stored BCrypt hash.
  - *Postcondition:* On success, an HTTP session is created and the user is redirected to `/dashboard`. On failure, the login page displays an error alert and no session is initialized.

Acceptance Criteria

1. The login page displays an “Email or username” field, a “Password” field, and a “Login” button within a centered card adhering to the application layout.

2. Username and email matching is case-insensitive, while password verification is strictly case-sensitive.
3. Submitting an incorrect password or an unknown username displays the same generic message: "Invalid email/username or password."
4. Empty form submissions are blocked client-side by browser validation.
5. Plaintext passwords are never logged, displayed, or stored; verification uses the database hash exclusively.

Test Scenarios

- **Manual (Success):** Open `/login`, enter `admin` and the admin password, and click Login. Verify the dashboard opens with "Welcome, Admin!". Repeat with `eldon@caracal.com` and its valid password.
- **Manual (Failure / Edge Case):** Enter `admin` with an incorrect password; verify the generic error alert appears. Enter a non-existent username; verify the identical message appears. Submit with an empty field and verify the browser blocks submission.
- **Automated (Unit / Slice / Integration):** JUnit 5 and Mockito test of `CustomUserDetailsService`: mocked repository returns a user for lookup `ADMIN` (case-insensitive), and an unknown lookup throws `UsernameNotFoundException`. A `MockMvc` test using `formLogin()` asserts redirect to `/dashboard` for valid credentials and to `/login?error=invalid` for invalid credentials.

Actionable Sub-tasks

- ☐ Create the `AppUser` JPA entity and `AppUserRepository` lookup methods by email or username (case-insensitive).
- ☐ Implement `CustomUserDetailsService` and `AppUserDetails`.
- ☐ Configure form login with success and failure handlers in `SecurityConfig`.
- ☐ Build `login.html` with error alert handling and shared styling.
- ☐ Write automated tests for `CustomUserDetailsService` and the login authentication flow.

Story 1.2: Role-Based Landing Page

- **GitHub Title:** `[Auth] Role-Based Landing Page`
- **Iteration:** `1`
- **Story Points:** `3`
- **Story Description:** As a signed-in user, I want a landing page that shows only the actions my role permits, so that I see relevant tools and avoid unauthorized options.

- **Actors / Personas:** Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**
  - *Precondition:* The user is authenticated.
  - *Trigger:* User logs in successfully or navigates directly to `/dashboard` or `/`.
- **Actions / Postconditions:**
  - *Action:* The application resolves the authenticated user's granted authorities and renders the view.
  - *Postcondition:* An `OPERATOR` sees a welcome message and "Change password". A `CLIENT_SERVICES` administrator also sees "Create account" and "Manage users".

Acceptance Criteria

1. Both roles see a welcome banner with their capitalized username and a top/sidebar bar containing a "Log out" trigger.
2. `OPERATOR` accounts see only the "Change password" quick-action button.
3. `CLIENT_SERVICES` accounts see "Create account", "Manage users", and "Change password".
4. The landing page never exposes internal database IDs, raw password hashes, or API header values.

Test Scenarios

- **Manual (Success):** Log in as `admin`; verify that all three action buttons ("Create account", "Manage users", "Change password") display. Log out, log in as `operator`, and verify only "Change password" displays.
- **Manual (Failure / Edge Case):** Log in as `operator`; inspect the DOM and verify that no "Create account" or "Manage users" links or buttons exist in the rendered markup.
- **Automated (Unit / Slice / Integration):** `@WebMvcTest` of `DashboardController` with a mocked `AppUserDetails` principal: assert model attribute `isAdmin=false` for `OPERATOR` and `isAdmin=true` for `CLIENT_SERVICES`, returning view name `dashboard` in both cases.

Actionable Sub-tasks

- ☐ Create `DashboardController` passing the username and administrative status flags to the model.
- ☐ Build `dashboard.html` with role-conditional action buttons using Thymeleaf `sec:authorize`.
- ☐ Build shared navigation top/sidebar bar in `fragments.html`.
- ☐ Write `@WebMvcTest` slice tests covering model attributes for both roles.

Story 1.3: Log Out

- **GitHub Title:** `[Auth] Log Out`
- **Iteration:** `1`
- **Story Points:** `1`
- **Story Description:** As a signed-in user, I want to log out, so that nobody else can use my session on a shared workstation.
- **Actors / Personas:** Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**
  - *Precondition:* The user has an active authenticated session.
  - *Trigger:* The user clicks "Log out" in the navigation bar.
- **Actions / Postconditions:**
  - *Action:* The browser submits an HTTP POST request to `/logout`. The server invalidates the session and deletes session cookies.
  - *Postcondition:* The user is redirected to `/login` with the alert "You have been logged out." and can no longer access protected pages.

Acceptance Criteria

1. Log out functionality is accessible across all protected application pages.
2. Following logout, the user is redirected to `/login?logout` displaying "You have been logged out."
3. Following logout, navigating directly to `/dashboard` redirects to `/login`.
4. Logout requires an HTTP POST request carrying a valid CSRF token to prevent cross-site logout exploits.

Test Scenarios

- **Manual (Success):** Log in and click "Log out". Verify the login page opens displaying "You have been logged out."
- **Manual (Failure / Edge Case):** Immediately after logging out, click the browser "Back" button or open `/dashboard` in a new tab. Verify the login page appears and no cached user data is displayed.
- **Automated (Unit / Slice / Integration):** `MockMvc`: a POST to `/logout` with a valid CSRF token as an authenticated user redirects to `/login?logout`. A POST without a CSRF token returns HTTP 403 Forbidden.

Actionable Sub-tasks

- ☐ Configure Spring Security logout handling (session invalidation, cookie deletion, success URL).
- ☐ Implement the Log out trigger as an HTTP POST form with CSRF token in `fragments.html`.
- ☐ Add the logout confirmation banner to `login.html`.
- ☐ Write `MockMvc` tests verifying session termination and CSRF enforcement.

Story 1.4: Automatic Logout After Inactivity

- **GitHub Title:** `[Auth] Automatic Logout After Inactivity`
- **Iteration:** `1`
- **Story Points:** `2`
- **Story Description:** As a Caracal administrator, I want user sessions to expire automatically after an inactivity period, so that unattended terminals cannot be misused.
- **Actors / Personas:** Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**
  - *Precondition:* A user is authenticated and stops interacting with the application.
  - *Trigger:* The user's next HTTP request arrives after the inactivity window has elapsed.
- **Actions / Postconditions:**
  - *Action:* The servlet container/Spring Session detects an expired session.
  - *Postcondition:* The request is redirected to `/login` and the user must sign in again.

Acceptance Criteria

1. The maximum inactivity limit is set to 30 minutes (`spring.session.timeout=30m`).
2. Sessions are persisted in PostgreSQL via Spring Session JDBC.
3. User interaction within the timeout window resets the expiration timer.
4. Application service restarts do not immediately log out active users, as sessions reside in the database rather than in-memory storage.

Test Scenarios

- **Manual (Success):** Temporarily set `spring.session.timeout=1m`, log in, wait 65 seconds without clicking anything, and refresh the page. Verify the login page appears with a session expired notification.
- **Manual (Failure / Edge Case):** Log in and navigate between pages every 20 seconds for two minutes. Verify the session remains active and the user is not signed out.
- **Automated (Unit / Slice / Integration):** `@SpringBootTest` verifying that the application context loads the `spring.session.timeout` property set to 30 minutes and initializes the Spring Session JDBC repository.

Actionable Sub-tasks

- ☐ Add `spring-boot-starter-session-jdbc` dependency to `pom.xml`.
- ☐ Configure `spring.session.jdbc.initialize-schema=always` and `spring.session.timeout=30m` in `application.properties`.
- ☐ Verify database migration generates the `SPRING_SESSION` and `SPRING_SESSION_ATTRIBUTES` tables.
- ☐ Write an automated configuration test verifying session timeout properties.

Story 1.5: Protected Pages and Login Redirects

- **GitHub Title:** `[Auth] Protected Pages and Login Redirects`
- **Iteration:** `1`
- **Story Points:** `2`
- **Story Description:** As an authenticated operator or administrator, I want unauthenticated visitors redirected to the login page and authenticated users redirected away from the login page, so that private transit data remains secure and authenticated users avoid redundant login forms.
- **Actors / Personas:** Unauthenticated Visitor, Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**
  - *Precondition:* None.
  - *Trigger:* A user accesses any application URL directly.
- **Actions / Postconditions:**
  - *Action:* The Spring Security filter chain evaluates the authentication status of the incoming request.
  - *Postcondition:* An unauthenticated visitor is redirected to `/login`. An already authenticated user accessing `/login` is redirected to `/dashboard`.

Acceptance Criteria

1. Only `/login`, static web resources (`/css/**`, `/js/**`, `/images/**`), and error views are publicly accessible.
2. Unauthenticated attempts to access `/dashboard`, `/admin/**`, or `/account/**` result in a 302 redirect to `/login`.
3. An authenticated user attempting to open `/login` is redirected to `/dashboard`.
4. Navigating to the root path `/` redirects authenticated users to `/dashboard` and unauthenticated users to `/login`.

Test Scenarios

- **Manual (Success):** While logged in, attempt to navigate to `/login`. Verify the application redirects immediately to `/dashboard`.
- **Manual (Failure / Edge Case):** In a private/incognito window while logged out, navigate directly to `/dashboard` and `/admin/users`. Verify both URLs redirect to `/login` and expose no internal data.
- **Automated (Unit / Slice / Integration):** `MockMvc`: anonymous `GET /dashboard` asserts HTTP 302 redirecting to `http://localhost/login`. Authenticated `GET /login` asserts HTTP 302 redirecting to `/dashboard`.

Actionable Sub-tasks

☐ Define HTTP request authorization matchers in `SecurityConfig`.
☐ Add redirect logic in `LoginController` to bypass `/login` when the security context is already authenticated.
☐ Ensure root path `/` forwards to `/dashboard`.
☐ Write `MockMvc` tests for anonymous redirects and authenticated bypasses.

Story 1.6: Restrict Administration Pages to Client Services

- **GitHub Title:** `[Auth] Restrict Administration Pages to Client Services`
- **Iteration:** `1`
- **Story Points:** `3`
- **Story Description:** As a Caracal administrator, I want account administration restricted to the `CLIENT_SERVICES` role, so that standard operators cannot provision users or elevate roles.
- **Actors / Personas:** Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**
  - *Precondition:* The user has an active authenticated session.

  - *Trigger:* The user accesses an administrative URL under `/admin/**`.

- **Actions / Postconditions:**
  - *Action:* The security interceptor checks whether the session principal holds the `CLIENT_SERVICES` authority.
  - *Postcondition:* A `CLIENT_SERVICES` user receives the page. An `OPERATOR` user receives HTTP 403 Forbidden with a styled Access Denied page.

Acceptance Criteria

1. All endpoints under `/admin/**` strictly require `ROLE_CLIENT_SERVICES`.
2. Unauthorized access by an `OPERATOR` returns HTTP 403 and renders `error/403.html` stating "You do not have permission to view this page."
3. The Access Denied view includes a navigation button returning to `/dashboard` matching the application theme.
4. Role checks are enforced on both HTTP GET views and HTTP POST mutation endpoints.

Test Scenarios

- **Manual (Success):** Log in as a `CLIENT_SERVICES` administrator and navigate to `/admin/users`. Verify the user management view renders correctly.
- **Manual (Failure / Edge Case):** Log in as an `OPERATOR` and type `/admin/users` or `/admin/users/new` directly into the browser address bar. Verify HTTP 403 and the Access Denied page display.
- **Automated (Unit / Slice / Integration):** `@WebMvcTest` of `AdminController`: `GET /admin/users` as `OPERATOR` returns HTTP 403, while as `CLIENT_SERVICES` it returns HTTP 200. A `POST /admin/users/new` as `OPERATOR` returns HTTP 403.

Actionable Sub-tasks

- ☐ Add `requestMatchers("/admin/**").hasRole("CLIENT_SERVICES")` in `SecurityConfig`.
- ☐ Create `templates/error/403.html` with return links and shared styling.
- ☐ Ensure administrative navigation buttons are hidden from `OPERATOR` views.
- ☐ Write `MockMvc` tests verifying HTTP 403 enforcement on GET and POST requests.

Story 1.7: View All Users

- **GitHub Title:** `[Auth] View All Users`
- **Iteration:** `1`

- **Story Points:** 2
- **Story Description:** As a Caracal administrator, I want to view every account with its username, email, and assigned role, so that I have complete visibility over who has access to the configuration tool.
- **Actors / Personas:** Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**
  - *Precondition:* The administrator is authenticated with `CLIENT_SERVICES`.
  - *Trigger:* The administrator navigates to `/admin/users`.
- **Actions / Postconditions:**
  - *Action:* The application queries all user records from the `app_users` table.
  - *Postcondition:* A table displays all accounts sorted alphabetically by username without modifying data.

Acceptance Criteria

1. The table contains Username, Email, and Role columns, sorted by username ascending.
2. Password hashes and credentials are never exposed in the HTML markup or API payloads.
3. The page provides a "Create account" button and a navigation link returning to the dashboard.
4. `OPERATOR` users are blocked from viewing this page (enforced via Story 1.6).

Test Scenarios

- **Manual (Success):** Log in as an administrator and click "Manage users". Verify all registered accounts appear in alphabetical order with their usernames, emails, and roles.
- **Manual (Failure / Edge Case):** Log in as an `OPERATOR` and attempt to navigate to `/admin/users`. Verify access is denied and no user roster data is leaked.
- **Automated (Unit / Slice / Integration):** JUnit and Mockito test of `UserService.listUsers()` verifying the repository is queried with sorting by username ascending. A `MockMvc` test asserts that `GET /admin/users` as `CLIENT_SERVICES` populates the `users` model attribute and returns the `users` view.

Actionable Sub-tasks

☐ Implement `UserService.listUsers()` sorting by username ascending.
☐ Create controller route `GET /admin/users` in `AdminController`.
☐ Build `templates/admin/users.html` table adhering to Bootstrap styling.

☐ Write automated service and controller slice tests.

Story 1.8: Change a User's Role

- **GitHub Title:** `[Auth] Change a User's Role`
- **Iteration:** `1`
- **Story Points:** `3`
- **Story Description:** As a Caracal administrator, I want to change another user's assigned role, so that staff members maintain only the permissions their current job function requires.
- **Actors / Personas:** Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**
  - *Precondition:* An administrator is signed in and the target user account exists in `app_users`.
  - *Trigger:* The administrator selects a new role in the user's table row and clicks "Save".
- **Actions / Postconditions:**
  - *Action:* The system validates the role selection, updates the user's database record, and terminates the target user's active sessions.
  - *Postcondition:* The table reflects the updated role. The target user must log in again to receive the new authority set.

Acceptance Criteria

1. Every user row except the currently logged-in administrator's contains a role selection dropdown and a "Save" button.
2. The active administrator's row displays their role as static text labeled with `(you)` and no dropdown.
3. The server rejects self-role modification attempts with the error message "You cannot change your own role."
4. Submitting an invalid role value displays "Please choose a valid role." Submitting for a non-existent user displays "That user no longer exists."
5. On success, an alert displays "Role updated. The user is signed out and receives the new role at their next login."

Test Scenarios

- **Manual (Success):** As an administrator, select a test operator, change their role to `CLIENT_SERVICES`, and click "Save". Verify the success banner appears and the table reflects the new role. If that user was signed in, verify their next page click forces them to log in again.

- **Manual (Failure / Edge Case):** Inspect the administrator's own row in `/admin/users`; verify there is no dropdown. Attempt to submit a role-change POST targeting the administrator's own ID; verify the server rejects the update with "You cannot change your own role."
- **Automated (Unit / Slice / Integration):** JUnit test of `UserService.changeRole()`: self-change returns a rejection status and does not save; invalid role returns an error; valid role saves and calls session invalidation. `MockMvc` POST to the role change endpoint as `CLIENT_SERVICES` with CSRF token asserts redirect to `/admin/users?roleChanged`.

Actionable Sub-tasks

- ☐ Implement `UserService.changeRole()` with self-modification prevention and input validation.
- ☐ Invalidate active sessions for the modified user via `FindByIndexNameSessionRepository`.
- ☐ Add `POST /admin/users/{id}/role` route in `AdminController`.
- ☐ Wire role dropdowns and forms in `templates/admin/users.html`.
- ☐ Write unit and slice tests for role modifications.

Story 1.9: Admin Creates an Account

- **GitHub Title:** `[Auth] Admin Creates an Account`
- **Iteration:** `1`
- **Story Points:** `3`
- **Story Description:** As a Caracal administrator, I want to provision new accounts with an email, username, role, and initial password, so that only approved personnel gain access to the system.
- **Actors / Personas:** Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**
  - *Precondition:* The administrator is authenticated with `CLIENT_SERVICES`.
  - *Trigger:* The administrator clicks "Create account", navigates to `/admin/users/new`, and submits the form.
- **Actions / Postconditions:**
  - *Action:* The system validates all input fields, encodes the password with BCrypt, and persists the record.
  - *Postcondition:* The new user appears in the user list and can immediately authenticate using their username or email.

Acceptance Criteria

1. The creation form includes Email, Username, Role (defaulting to `OPERATOR`), Password, and Confirm Password fields.
2. Email must be a valid RFC 5322 format. Username must be 3–30 characters containing only alphanumeric characters, dots, underscores, or hyphens.
3. Password and confirmation must match and satisfy the security policy defined in Story 1.11.
4. Email and username are stored in lowercase. Passwords are saved strictly as BCrypt hashes.
5. On validation failure, the form re-renders displaying field error messages while preserving entered email, username, and role (clearing password fields).
6. Success redirects to `/admin/users` displaying "Account created."
7. Public user self-registration is disabled; accounts are created exclusively by administrators.

Test Scenarios

- **Manual (Success):** As an administrator, create account `testoperator` with email `test.operator@nywaterway.com`, role `OPERATOR`, and password `Ferry@2026`. Log out, then log in using the username and then the email; verify both authentications succeed.
- **Manual (Failure / Edge Case):** Submit mismatched passwords, a username containing spaces, and an email without a domain. Verify individual error alerts display ("Passwords do not match.") and the typed username and email are preserved.
- **Automated (Unit / Slice / Integration):** JUnit test of `UserService.createUser()`: valid input saves an `AppUser` with normalized lowercase username/email and hashed password; invalid input throws validation exceptions. `MockMvc` POST with invalid data asserts view `admin/create-account` is returned with an `errors` model attribute.

Actionable Sub-tasks

- ☐ Implement `CreateUserForm` DTO with Bean Validation constraints.
- ☐ Implement `UserService.createUser()` with BCrypt hashing and normalization.
- ☐ Add `GET /admin/users/new` and `POST /admin/users/new` in `AdminController`.
- ☐ Build `templates/admin/create-account.html` with field error feedback.
- ☐ Write automated tests for user provisioning and validation rejection.

Story 1.10: Prevent Duplicate Usernames and Emails

- **GitHub Title:** `[Auth] Prevent Duplicate Usernames and Emails`

- **Iteration:** `1`
- **Story Points:** `2`
- **Story Description:** As a Caracal administrator, I want the system to reject duplicate usernames and emails, so that every account uniquely identifies exactly one staff member.
- **Actors / Personas:** Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**
  - *Precondition:* At least one account exists in `app_users`.
  - *Trigger:* An administrator submits `/admin/users/new` with a username or email that already exists.
- **Actions / Postconditions:**
  - *Action:* The system checks uniqueness case-insensitively, and database unique constraints enforce uniqueness at the storage layer.
  - *Postcondition:* The transaction is aborted, no account is inserted, and the form informs the user which field is duplicated.

Acceptance Criteria

1. A duplicate username displays "That username is already taken."
2. A duplicate email displays "That email is already registered."
3. Comparisons are case-insensitive (e.g., `admin` and `ADMIN` are treated as identical).
4. Concurrent submissions with identical values are caught by database constraints, presenting a safe error message ("That username or email is already in use.") without exposing SQL stack traces.
5. Usernames cannot contain the `@` character, preventing collisions between usernames and emails.

Test Scenarios

- **Manual (Success):** Create an account with a unique username and email. Verify the account is created successfully.
- **Manual (Failure / Edge Case):** Attempt to create an account with username `admin`, then `ADMIN`, then email `admin@caracal.com`. Verify each submission is rejected with its corresponding duplicate notification.
- **Automated (Unit / Slice / Integration):** JUnit test: when `existsByUsernameIgnoreCase` or `existsByEmailIgnoreCase` returns true, `createUser` returns a duplicate error and aborts persistence. When `saveAndFlush` throws `DataIntegrityViolationException`, the service safely catches it and returns "That username or email is already in use."

Actionable Sub-tasks

- ☐ Add `existsByUsernameIgnoreCase` and `existsByEmailIgnoreCase` to `AppUserRepository`.
- ☐ Apply unique constraints to `username` and `email` columns in PostgreSQL schema.
- ☐ Add duplicate pre-checks and `DataIntegrityViolationException` handling in `UserService`.
- ☐ Write unit tests verifying duplicate rejection for both usernames and emails.

Story 1.11: Password Rules

- **GitHub Title:** `[Auth] Enforce Password Rules`
- **Iteration:** `1`
- **Story Points:** `2`
- **Story Description:** As an administrator, I want system passwords to meet strength requirements, so that accounts are protected from brute-force and dictionary guessing.
- **Actors / Personas:** Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**
  - *Precondition:* A user password is being established.
  - *Trigger:* An administrator submits "Create account", or any user submits "Change password".
- **Actions / Postconditions:**
  - *Action:* The server validates the candidate password against the complexity policy.
  - *Postcondition:* A password satisfying all rules is accepted. An invalid password is rejected and all failing criteria are presented together.

Acceptance Criteria

1. Length must be between 8 and 72 characters inclusive.
2. Must contain at least one uppercase letter.
3. Must contain at least one lowercase letter.
4. Must contain at least one special symbol (e.g., `! @ # $ % ^ & *`).
5. All violated rules are returned simultaneously in a structured feedback list.
6. Complexity is enforced server-side, supplemented by client-side form hint text.

Test Scenarios

- **Manual (Success):** Enter `Ferry@2026` as a new password. Verify it is accepted without errors.
- **Manual (Failure / Edge Case):** Submit `ferry@2026` (missing uppercase), `FERRY@2026` (missing lowercase), `Ferry2026x` (missing symbol), and `abcdefgh` (multiple failures). Verify each attempt displays the exact list of missing requirements.
- **Automated (Unit / Slice / Integration):** JUnit 5 parameterized test for `PasswordPolicy.validate()` testing a matrix of valid, weak, short, and 73+ character passwords against expected error outputs.

Actionable Sub-tasks

☐ Implement `PasswordPolicy` validation utility class.
☐ Integrate password policy checks into `CreateUserForm` and `ChangePasswordForm`.
☐ Add policy hint text and live client-side validation hints to password views.
☐ Write parameterized JUnit tests covering all policy branches.

Story 1.12: Change My Own Password

- **GitHub Title:** `[Auth] Change My Own Password`
- **Iteration:** `1`
- **Story Points:** `3`
- **Story Description:** As an authenticated user, I want to change my own password by verifying my current one, so that I can maintain my credentials independently without administrative intervention.
- **Actors / Personas:** Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**
  - *Precondition:* The user is logged in.
  - *Trigger:* The user clicks "Change password", navigates to `/account/password`, and submits the form.
- **Actions / Postconditions:**
  - *Action:* The system verifies the current password hash, validates the new password against Story 1.11 rules, and stores the new BCrypt hash.
  - *Postcondition:* The old password is invalidated, and the new password is required for subsequent logins.

Acceptance Criteria

1. Form requires Current Password, New Password, and Confirm New Password.
2. Entering an incorrect current password displays "Current password is incorrect." and leaves credentials unchanged.
3. Mismatched new passwords display "New passwords do not match."
4. Entering a new password identical to the current password displays "New password must be different from the current password."
5. New passwords must satisfy all rules from Story 1.11.
6. Success redirects to `/dashboard` displaying "Your password has been updated."
7. Users can only modify their own credentials; target user ID cannot be overridden via request tampering.

Test Scenarios

- **Manual (Success):** Log in with a test account, navigate to `/account/password`, enter current credentials, and set new password to `Harbor#2026`. Verify success notification appears, log out, and log in with the new password.
- **Manual (Failure / Edge Case):** Enter an invalid current password, then mismatched new passwords, then reuse the current password. Verify each scenario displays its respective error message and leaves stored credentials intact.
- **Automated (Unit / Slice / Integration):** JUnit test of `UserService.changePassword()` with `BCryptPasswordEncoder`: valid current password replaces the hash; incorrect current password returns an error status and aborts save. `MockMvc` POST as an authenticated user asserts redirect to `/dashboard?passwordChanged`.

Actionable Sub-tasks

- ☐ Implement `ChangePasswordForm` DTO with validation.
- ☐ Add `UserService.changePassword()` verifying existing hash and hashing new password.
- ☐ Create `GET /account/password` and `POST /account/password` in `AccountController`.
- ☐ Build `templates/account/change-password.html` with field error indicators.
- ☐ Write unit and controller slice tests for password updates.

Story 1.13: Store Passwords as Hashes

- **GitHub Title:** `[Auth] Hash Stored Passwords`

- **Iteration:** `1`
- **Story Points:** `1`
- **Story Description:** As a Caracal administrator, I want passwords stored strictly as salted one-way hashes, so that a database breach does not expose user credentials.
- **Actors / Personas:** Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**
  - *Precondition:* A password is created or updated.
  - *Trigger:* Account provisioning, self-service password update, or initial database seeding.
- **Actions / Postconditions:**
  - *Action:* The application passes the plaintext password through `BCryptPasswordEncoder` prior to writing to storage.
  - *Postcondition:* The `password_hash` database column stores a BCrypt string starting with `$2a$` or `$2b$`. Plaintext passwords are never persisted.

Acceptance Criteria

1. All stored passwords in `app_users.password_hash` are formatted as valid BCrypt hashes.
2. Plaintext passwords never appear in application log files, database dumps, or UI templates.
3. Authentication evaluates hashes via `PasswordEncoder.matches()`. Entering a pre-computed hash into the password input does not authenticate.

Test Scenarios

- **Manual (Success):** Query the `app_users` table in PostgreSQL. Verify all `password_hash` values begin with `$2` and contain no readable text.
- **Manual (Failure / Edge Case):** Copy a valid BCrypt hash from the database and paste it directly into the password field on `/login`. Verify authentication fails with "Invalid email/username or password."
- **Automated (Unit / Slice / Integration):** JUnit test: asserting that after `createUser`, the persisted `passwordHash` does not equal the raw input, starts with `$2`, and evaluates to `true` when tested with `passwordEncoder.matches(raw, hash)`.

Actionable Sub-tasks

- ☐ Declare `BCryptPasswordEncoder` bean in `SecurityConfig`.

- ☐ Verify hashing is applied in user provisioning, password updates, and test data seeding.
- ☐ Write automated tests asserting hash formatting and matching behavior.

Story 1.14: Google Login

- **GitHub Title:** `[Auth] Google Login`
- **Iteration:** `2`
- **Story Points:** `3`
- **Story Description:** As a staff member, I want to sign in with my authorized Google account, so that I can access the configuration tool quickly without managing a separate local password.
- **Actors / Personas:** Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**
  - *Precondition:* The user's Google email exists in `app_users` and a Google OAuth2 client is registered.
  - *Trigger:* The user clicks "Sign in with Google" on the login page.
- **Actions / Postconditions:**
  - *Action:* The user authenticates with Google, and the application matches the verified email claim to an existing `app_users` record.
  - *Postcondition:* The user is redirected to the role-based dashboard matching their database permissions.

Acceptance Criteria

1. The `/login` page offers a "Sign in with Google" button alongside the standard username/password form.
2. Authentication succeeds only if the Google email already exists in `app_users`; system roles are resolved from the local database, never from external IdP claims.
3. Authenticating with an unrecognized Google account displays an access denied message ("Your Google account is not authorized to access this system") and creates no database record.
4. Session timeout, inactivity handling, and logout behavior function identically to password-based sessions.

Test Scenarios

- **Manual (Success):** Click "Sign in with Google" using an authorized email present in `app_users`. Verify the application signs in and displays the expected role landing page.
- **Manual (Failure / Edge Case):** Sign in with an unauthorized Google email not present in `app_users`. Verify authentication is rejected and the not-authorized error banner appears.
- **Automated (Unit / Slice / Integration):** Unit test of OAuth2 user mapping service: mocked repository resolving a known email constructs an authorized `AppUserDetails` principal; an unknown email throws `OAuth2AuthenticationException`.

Actionable Sub-tasks

- ☐ Add `spring-boot-starter-oauth2-client` dependency.
- ☐ Configure Google OAuth2 client properties in `application.properties`.
- ☐ Implement `CustomOAuth2UserService` mapping external email claims to local `app_users` records.
- ☐ Add Google sign-in button to `login.html`.
- ☐ Write automated lookup tests for authorized and unauthorized Google identities.

Story 1.15: Operator Identity on Caracal API Requests

- **GitHub Title:** `[Auth] Send Operator Identity in X-Actor Header`
- **Iteration:** `2`
- **Story Points:** `2`
- **Story Description:** As a Caracal administrator, I want every mutating request sent to Caracal's platform API to include the authenticated operator's email in the `X-Actor` header, so that all configuration changes are attributable in the platform audit log.
- **Actors / Personas:** Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**
  - *Precondition:* The user is logged in and an active application module prepares an HTTP request to Caracal's API.
  - *Trigger:* The user performs a mutating action (e.g., updating a fare matrix or saving an alert).
- **Actions / Postconditions:**
  - *Action:* The application's HTTP client intercepts the outgoing request and attaches `X-Actor: operator:<email>`.

– *Postcondition:* Caracal's platform API processes the request and writes the operator's email into the platform audit trail.

Acceptance Criteria

1. The outgoing request header is formatted as `X-Actor: operator:<email>`, where `<email>` is the verified email of the authenticated session.
2. The header value is derived server-side from the security context and cannot be spoofed by client input.
3. Any API request initiated without an authenticated session context is aborted before reaching the network.

Test Scenarios

- **Manual (Success):** Log in as `operator@nywaterway.com`, perform a rate change, and verify via network inspection/audit log that the outgoing API call carries `X-Actor: operator:operator@nywaterway.com`.
- **Manual (Failure / Edge Case):** Attempt to invoke the API client service without an active security context. Verify an `IllegalStateException` is thrown and no external network request is dispatched.
- **Automated (Unit / Slice / Integration):** JUnit test of `ActorService.currentActorHeader()`: asserting it returns `operator:<email>` when an authenticated principal exists and throws `IllegalStateException` when unauthenticated. WireMock test asserting the `X-Actor` header is present on outgoing REST calls.

Actionable Sub-tasks

- ☐ Implement `ActorService.currentActorHeader()` resolving email from `SecurityContextHolder`.
- ☐ Configure `WebClient` / `RestTemplate` interceptor injecting `X-Actor` on outgoing Caracal requests.
- ☐ Write unit tests for `ActorService` under authenticated and unauthenticated states.
- ☐ Write WireMock integration tests verifying header attachment on simulated API calls.

Epic 2: Interactive Origin-Destination Rate Grid & Fare Management

This document defines the formal user stories and sub-stories for Epic 2 of the NY Waterway Fare Configuration Tool. Each story adheres strictly to the CMPT 276 agile project specification, INVEST criteria, and standardized persona definitions.

Story 2.1: Unified Tenant Context Switching & Dual-Matrix View Scaffolding

- **GitHub Issue Title:** `[Rate Grid] Unified Tenant Context Switching & Dual-Matrix View Scaffolding`
- **Iteration:** `1`
- **Story Points:** `3`
- **Story Description:** As a Caracal administrator, I want the matrix grid to automatically adapt its axes and data bindings based on the selected tenant context, so that I can manage point-to-point ferry routes (NY Waterway) or zone-based rail networks (Metra) within a single unified application.
- **Actors / Personas:** Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**
  - *Precondition:* Operator is authenticated and navigates to the Rate Management view (`/fares`).
  - *Trigger:* The user selects a tenant from the global context dropdown (e.g., `NYW_SANDBOX` or `METRA`).
- **Actions / Postconditions:**
  - *Action:* The user toggles between tenant contexts in the navigation bar.
  - *Postcondition:* Selecting `NYW_SANDBOX` binds the view to **Point-to-Point (O-D) Mode** (origin terminal selector, outbound destination rows, ticket product columns). Selecting `METRA` binds the view to **Zonal Matrix Mode** (fare product selector, origin zone rows, destination zone columns). Cells display formatted rates (`$XX.XX`) or distinct badges (`NOT_SOLD`, `N/A`).

Acceptance Criteria

1. Switching the tenant dropdown dynamically re-renders the grid structure to the corresponding axis mode without a full-page error reload.
2. In O-D Mode, changing the origin terminal dynamically updates outbound destination rows and fetches current route prices.
3. In Zonal Mode, changing the active ticket product dynamically updates the zone-to-zone fare intersections.
4. Every cell without an active price rule resolves to a muted `NOT_SOLD` or `N/A` badge; no cell renders as blank, `$0.00`, or `NaN`.

Test Scenarios

- **Manual (Success):** Select `NYW_SANDBOX` → select `EDGEWTR` terminal. Verify rows show destinations (`MIDTOWN`, `BROOKFD`) and columns show

products (ADULT_ONEWAY). Switch tenant to METRA → select ADULT_ONEWAY. Verify the table renders a symmetric Zone × Zone grid with rates formatted as $XX.XX.

- **Manual (Failure / Edge Case):** Select a terminal with no outbound routes or an unconfigured zone pair. Verify the cell displays a gray NOT_SOLD badge rather than breaking table formatting or rendering blank.
- **Automated (Unit / Slice / Integration):** Execute a MockMvc GET /fares?tenant=NYW_SANDBOX&origin=EDGEWTR asserting HTTP 200, verifying model attribute matrixMode=OD and table cells in the Thymeleaf response; execute GET /fares?tenant=METRA&product=ADULT_ONEWAY asserting HTTP 200 and matrixMode=ZONAL.

Actionable Sub-tasks

- ☐ Implement TenantContext model and register tenant session resolver in Spring Boot.
- ☐ Create Thymeleaf layout with responsive context toolbar (Tenant dropdown, Mode indicator, Terminal/Product secondary selector).
- ☐ Build dynamic matrix table component in rate-grid.html supporting both destination and zone row/column iteration.
- ☐ Define CSS styling for price cells, NOT_SOLD badges, and N/A muted pills.
- ☐ Write controller slice tests (@WebMvcTest) verifying dual-mode model attributes.

Story 2.2: O-D Mode Direct Inline Rate Editing & Numeric Input Masking

- **GitHub Issue Title:** [Rate Grid] O-D Mode Direct Inline Rate Editing & Numeric Input Masking
- **Iteration:** 2
- **Story Points:** 3
- **Story Description:** As an authenticated operator, I want to click directly into a price cell in the O-D grid to edit the rate inline with real-time numeric validation, so that I can update point-to-point fares quickly and prevent malformed or negative amounts from being entered.
- **Actors / Personas:** Authenticated Operator (OPERATOR), Caracal Administrator (CLIENT_SERVICES)
- **Preconditions & Triggers:**
  - *Precondition:* The application is in O-D Mode (NYW_SANDBOX), displaying an active terminal rate table.
  - *Trigger:* The user clicks on any active price cell.

- **Actions / Postconditions:**
  - *Action:* The static currency text converts into a focused numeric input field (`<input type="number">`). Keystrokes are sanitized in real time.
  - *Postcondition:* Pressing Enter or blurring the input commits the value to local draft state, while pressing Escape cancels the edit.

Acceptance Criteria

1. Clicking a price cell converts the cell text into an active input pre-populated with the unformatted numeric value.
2. Input masking restricts entries strictly to positive numbers with a maximum of two decimal places (`min="0.00"`, `step="0.01"`).
3. Negative signs, alphabetic letters, and third decimal points are blocked on keystroke.
4. Entering a blank value or invalid string applies the `.is-invalid` visual class, disables the save trigger, and alerts the user.
5. Cells flagged as `NOT_SOLD` cannot be edited directly via inline text without explicitly toggling their sold state.

Test Scenarios

- **Manual (Success):** Click `EDGEWTR -> MIDTOWN` Adult fare (`$13.25`), change to `14.00`, and press Enter. Verify the cell reflects `$14.00`.
- **Manual (Failure / Edge Case):** Click a price cell, attempt to type `-12.50`, `abc`, or `13.999`. Verify that invalid characters are blocked on keystroke and the input highlights in red if forced.
- **Automated (Unit / Slice / Integration):** Jest/frontend unit test verifying cell input event handlers sanitize raw strings and reject values failing `/^\d+(\.\d{1,2})?$/`.

Actionable Sub-tasks

- ☐ Implement client-side inline edit toggle (`click` converts span to input, `blur`/`Enter` converts back to span).
- ☐ Add vanilla JS input masking to reject negative numbers, letters, and excessive decimal places.
- ☐ Add `.is-invalid` error styling and accessible ARIA attributes for malformed inputs.
- ☐ Lock `NOT_SOLD` and unconfigured cells from inline text entry.
- ☐ Write automated browser tests verifying keystroke validation rules.

Story 2.3: Zonal Cell Action Modal for Rate Tier Management (Parent Story)

- **GitHub Issue Title:** `[Rate Grid] Zonal Cell Action Modal for Rate Tier Management`
- **Iteration:** `2`
- **Story Points:** `5` *(Split into sub-stories 2.3a and 2.3b)*
- **Story Description:** As a Caracal administrator in Zonal Mode, I want clicking a matrix cell to open a Rate Action Modal with options to update the shared tier globally, reassign the cell to an existing tier, or create a new tier, so that I can manage network-wide zonal fare structures accurately.
- **Actors / Personas:** Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**
  - *Precondition:* The application is in Zonal Mode (`METRA`).
  - *Trigger:* The user clicks on any zone-to-zone cell in the grid.
- **Actions / Postconditions:**
  - *Action:* The operator selects one of three rate management actions or toggles availability.
  - *Postcondition:* The modal closes and stages the selected rate adjustment into local draft state with metadata indicating the update scope.

Acceptance Criteria

1. The modal displays three distinct action options: Global Rate Tier Adjustment (Option 1), Reassign to Existing Tier (Option 2), and Create & Assign New Tier (Option 3).
2. Option 1 explicitly warns the user how many total network zone pairs share this tier and will be changed.
3. Option 2 provides a dropdown of existing active system tiers.
4. Option 3 validates currency input strictly (`> 0.00`, two decimal places).
5. The modal includes a toggle to switch the cell between active pricing and `NOT_SOLD`.

Test Scenarios

- **Manual (Success):** Click a `$4.90` cell → select Option 1 → enter `$5.00` → apply. Verify all cells sharing that tier in the matrix reflect `$5.00` and display draft highlights.
- **Manual (Failure / Edge Case):** Select Option 3 → leave the new rate field blank or enter a negative value (`-3.00`). Verify validation prevents applying the change and displays an inline error alert.

- **Automated (Unit / Slice / Integration):** Trigger modal submission via JavaScript test fixture; verify that the emitted change event payload contains the correct `actionType` (`GLOBAL_UPDATE`, `REASSIGN_TIER`, or `NEW_TIER`) and target rate.

Actionable Sub-tasks

- ☐ Construct Bootstrap 5 modal markup for the Zonal Rate Action dialog.
- ☐ Implement dynamic modal population showing current pair (`originZone`, `destZone`) and linked tier details.
- ☐ Implement Option 1 form controls with dynamic warning text showing count of affected pairs.
- ☐ Implement Option 2 dropdown populated with active rate tier entities.
- ☐ Implement Option 3 form controls with currency validation for new tier creation.
- ☐ Wire modal "Apply" button to update grid cell display and dispatch draft state events.

Story 2.3a: Zonal Action Modal UI & Tier Selection Dropdown

- **GitHub Issue Title:** `[Rate Grid] Zonal Action Modal UI & Tier Selection Dropdown`
- **Iteration:** `2`
- **Story Points:** `2`
- **Story Description:** As an authenticated operator in Zonal Mode, I want clicking a matrix cell to open an action modal displaying the active zone pair and a dropdown of existing rate tiers, so that I can reassign that route to a different existing price tier.
- **Actors / Personas:** Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**
  - *Precondition:* The application is in Zonal Mode (`METRA`).
  - *Trigger:* The operator clicks any zone-to-zone cell in the grid.
- **Actions / Postconditions:**
  - *Action:* A modal opens showing origin and destination zone labels alongside current pricing. The user selects an existing tier from the dropdown (Option 2) and clicks "Apply".
  - *Postcondition:* The clicked cell updates its staged rate in local draft state, and the modal closes without reloading the page.

Acceptance Criteria

1. Clicking any active zonal cell opens the modal without a page reload.
2. Modal displays the origin zone, destination zone, and active fare.
3. Dropdown lists all active system rate tiers with their dollar values.
4. Selecting a tier and clicking "Apply" updates only the clicked cell's staged rate.
5. Includes an availability toggle to flag the cell as `NOT_SOLD`.

Test Scenarios

- **Manual (Success):** Click a `$5.90` cell → select `$7.40` tier from dropdown → apply. Verify only that single cell updates to `$7.40` in the grid.
- **Manual (Failure / Edge Case):** Open the Rate Action Modal for an active zone pair, leave the tier selection on the unselected placeholder (or choose an empty/invalid value), and click "Apply". Verify an inline error message ("Please select a valid rate tier") appears, the modal remains open, and the cell value in the matrix does not change.
- **Automated (Unit / Slice / Integration):** JavaScript unit test verifying the modal open event populates tier options and emits a `REASSIGN_TIER` event on success; assert submitting with a null/empty `tierId` blocks submission, prevents event dispatch, and applies `.is-invalid` to the dropdown.

Actionable Sub-tasks

☐ Build modal shell and backdrop in `rate-grid.html`.
☐ Wire cell click listener to pass selected `originZone` and `destZone` to modal view state.
☐ Implement Option 2 select element populated with system rate tiers.
☐ Add availability switch for toggling `NOT_SOLD`.
☐ Write unit test for modal form submission validation.

Story 2.3b: Global Tier Adjustment & New Tier Creation Logic

- **GitHub Issue Title:** `[Rate Grid] Global Tier Adjustment & New Tier Creation Logic`
- **Iteration:** `2`
- **Story Points:** `3`
- **Story Description:** As a Caracal administrator, I want the zonal rate modal to support modifying a tier's base price globally or creating a brand new tier, so that I can adjust shared network rates or define custom fare levels.

- **Actors / Personas:** Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**
  - *Precondition:* The Rate Action Modal is open for a selected cell.
  - *Trigger:* The user selects either Option 1 (Global Rate Update) or Option 3 (Create New Tier).
- **Actions / Postconditions:**
  - *Action:* The user modifies a tier's base value or inputs a custom price for a new tier.
  - *Postcondition:* Choosing Option 1 updates the shared tier price, flagging all linked network cells as modified. Choosing Option 3 registers a new tier and links the active cell to it.

Acceptance Criteria

1. Option 1 displays an impact warning stating the total count of network zone pairs sharing that tier.
2. Applying Option 1 updates all matrix cells associated with that tier in draft state.
3. Option 3 validates currency input strictly (`> 0.00`, two decimal places).
4. Option 3 creates the new tier and assigns it exclusively to the active cell.

Test Scenarios

- **Manual (Success):** Select Option 1 for a `$4.90` tier (12 linked pairs) → enter `$5.00` → apply. Verify all 12 cells update to `$5.00`. Select Option 3 → enter `$6.50` → verify a new tier is created and applied to the cell.
- **Manual (Failure / Edge Case):** Select Option 1 (Global Update) or Option 3 (New Tier) and input a negative number (`-$1.00`), blank text, or a third decimal place (`$5.555`). Click "Apply" and verify an inline validation error displays ("Price must be a positive number with at most two decimal places"), input border highlights red, and no matrix cells or global tier values are modified.
- **Automated (Unit / Slice / Integration):** Frontend unit test asserting `GLOBAL_UPDATE` and `NEW_TIER` payload construction on valid input; assert that negative or non-decimal inputs fail currency regex validation and do not emit change events.

Actionable Sub-tasks

☐ Add Option 1 radio button, input field, and dynamic affected-pairs calculation.
☐ Add Option 3 radio button and validated new tier input field.
☐ Wire event handlers to broadcast global vs. single-cell draft updates.
☐ Write unit tests verifying input regex validation for new tier creation.

Story 2.4: Global Rate Tier Catalog & Bulk Adjustment Tool (Parent Story)

- **GitHub Issue Title:** `[Rate Grid] Global Rate Tier Catalog & Bulk Adjustment Tool`
- **Iteration:** `2`
- **Story Points:** `5` *(Split into sub-stories 2.4a and 2.4b)*
- **Story Description:** As a Caracal administrator, I want a dedicated catalog view of all active rate tiers with a bulk adjustment tool, so that I can execute network-wide fare changes (such as an annual 10% increase) across all tiers simultaneously.
- **Actors / Personas:** Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**
  - *Precondition:* Operator is in Zonal Mode (`METRA`).
  - *Trigger:* The user clicks the "Manage Rate Tiers" button in the header toolbar.
- **Actions / Postconditions:**
  - *Action:* A management panel displays a summarized table of all active rate tiers, their values, and their assignment counts. The operator applies a percentage or fixed dollar increase across all tiers.
  - *Postcondition:* The operator previews recalculations and stages bulk adjustments across all affected matrix cells.

Acceptance Criteria

1. Displays every active rate tier with: Tier Code/Name, Base Rate ($), and Number of Linked Station/Zone Pairs.
2. The bulk adjustment tool supports percentage increases (rounded to the nearest cent) and flat increments.
3. Clicking "Preview Bulk Adjustment" shows side-by-side Old Price vs. New Price comparisons for every tier before applying.
4. Applying updates stages modifications for all affected tiers and marks corresponding matrix cells as drafted.

Test Scenarios

- **Manual (Success):** Open Rate Tier Manager → input `10%` increase → click Preview. Verify a `$4.90` tier becomes `$5.39` and a `$7.40` tier becomes `$8.14`. Click "Apply to Grid" and verify the active zonal matrix reflects the updated values.

- **Manual (Failure / Edge Case):** Input a negative percentage that would drop rates below `$0.00` (e.g., `-150%`) or non-numeric input. Verify validation blocks the calculation and displays an inline error alert ("Adjustment must result in fares greater than $0.00").
- **Automated (Unit / Slice / Integration):** JUnit service test for `RateTierService.calculateBulkAdjustment(List<RateTier> tiers, AdjustmentType type, double value)` asserting correct mathematical half-up rounding and boundary checks; assert `IllegalArgumentException` on negative-resulting rates.

Actionable Sub-tasks

- ☐ Create `RateTier` DTO and in-memory/database catalog registry service.
- ☐ Build "Manage Rate Tiers" modal / view with tier summary table.
- ☐ Implement bulk calculation engine in JavaScript/Java supporting percentage and flat addition with half-up cent rounding.
- ☐ Add side-by-side preview diff table inside the tier manager dialog.
- ☐ Write unit tests verifying precision rounding and negative-rate prevention.

Story 2.4a: Rate Tier Catalog View & Summary Table

- **GitHub Issue Title:** `[Rate Grid] Rate Tier Catalog View & Summary Table`
- **Iteration:** `2`
- **Story Points:** `2`
- **Story Description:** As a Caracal administrator, I want a dedicated catalog view displaying all active rate tiers and their usage counts, so that I have complete visibility over network fare groupings.
- **Actors / Personas:** Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**
  - *Precondition:* Operator is in Zonal Mode (`METRA`).
  - *Trigger:* The user clicks the "Manage Rate Tiers" button in the toolbar.
- **Actions / Postconditions:**
  - *Action:* The user opens the tier catalog view.
  - *Postcondition:* A management dialog displays a consolidated table of all active rate tiers, their base currency values, and the total count of mapped zone pairs.

Acceptance Criteria

1. Table lists Tier Code, Base Fare ($), and Number of Linked Station/Zone Pairs.
2. Unassigned or empty tiers are clearly labeled.
3. Table dynamically reflects in-memory draft changes made during the active session.
4. If no tiers exist or if service retrieval fails, a clear fallback notification is rendered.

Test Scenarios

- **Manual (Success):** Click "Manage Rate Tiers". Verify the table matches active system tiers and accurately totals mapped zone pairs.
- **Manual (Failure / Edge Case):** Click "Manage Rate Tiers" when no rate tiers exist in the system or when the service encounters a backend fetch failure. Verify the modal displays a clear alert banner ("No active rate tiers found; please configure a tier" or "Unable to load rate tiers; please refresh") and disables the bulk adjustment controls rather than crashing or rendering a broken table.
- **Automated (Unit / Slice / Integration):** Spring MVC test (`@WebMvcTest`) asserting the endpoint returns HTTP 200 with active `RateTier` DTO collections; simulate service failure asserting HTTP 500 or error view with a user-friendly error message.

Actionable Sub-tasks

☐ Define `RateTierDto` class with tier code, rate value, and pair count fields.
☐ Create controller endpoint `GET /api/rate-tiers` returning active tier summaries.
☐ Build modal view with Bootstrap summary table.
☐ Implement empty state and error state alert templates.
☐ Write `@WebMvcTest` controller test for tier catalog retrieval.

Story 2.4b: Bulk Percentage & Flat Fare Adjustment Engine

- **GitHub Issue Title:** `[Rate Grid] Bulk Percentage & Flat Fare Adjustment Engine`
- **Iteration:** `2`
- **Story Points:** `3`
- **Story Description:** As a Caracal administrator, I want to calculate and apply bulk percentage or flat adjustments across all rate tiers simultaneously, so that I can execute system-wide fare updates in a single action.

- **Actors / Personas:** Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**
  - *Precondition:* The "Manage Rate Tiers" dialog is open.
  - *Trigger:* The user inputs an adjustment value and clicks "Preview Bulk Adjustment".
- **Actions / Postconditions:**
  - *Action:* The user inputs a percentage (e.g., `+10%`) or flat increment (e.g., `+$0.25`), reviews Old vs. New comparisons, and clicks "Apply to Grid".
  - *Postcondition:* The application recalculates rates and stages draft modifications for all affected tiers across the matrix.

Acceptance Criteria

1. Supports percentage (`%`) and fixed dollar (`$`) calculations.
2. Applies standard financial half-up rounding to the nearest cent (`$0.01`).
3. Displays a side-by-side Old Price vs. New Price diff preview before committing.
4. Blocks calculations resulting in rates ≤ 0.00.
5. Applying recalculates and stages draft changes for all affected matrix cells.

Test Scenarios

- **Manual (Success):** Input `10%` increase $\rightarrow$ click Preview. Verify `$4.90` becomes `$5.39` and `$7.40` becomes `$8.14`. Click Apply and verify matrix cells update.
- **Manual (Failure / Edge Case):** In the bulk adjustment tool, enter an extreme negative percentage that drops rates to or below `$0.00` (e.g., `-120%`) or enter non-numeric text (`abc`) and click "Preview". Verify an inline alert appears ("Adjustment must result in fares greater than $0.00"), preview diffs do not calculate, and the "Apply to Grid" button remains disabled.
- **Automated (Unit / Slice / Integration):** JUnit service test verifying `calculateBulkAdjustment()` rounds cents accurately (half-up) on valid input; assert that method throws `IllegalArgumentException` when an adjustment results in fares ≤ 0.00.

Actionable Sub-tasks

- ☐ Implement `calculateBulkAdjustment` service logic with `BigDecimal` half-up rounding.
- ☐ Build preview diff table component showing Old Price, New Price, and Absolute Change.

☐ Wire "Apply to Grid" button to update client-side matrix cells and trigger draft state.
☐ Add client-side validation blocking invalid or non-numeric adjustment inputs.
☐ Write JUnit unit tests covering flat, percentage, and boundary rounding edge cases.

Story 2.5: Station-to-Zone Topology Mapping Side Panel

- **GitHub Issue Title:** `[Rate Grid] Station-to-Zone Topology Mapping Side Panel`
- **Iteration:** `2`
- **Story Points:** `3`
- **Story Description:** As a Caracal administrator managing a zonal rail network, I want to view and reassign stations to zones using a slide-out drawer on the same screen as the matrix, so that I can maintain station zone assignments without navigating away from the fare table.
- **Actors / Personas:** Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**
  - *Precondition:* User is in Zonal Mode (`METRA`).
  - *Trigger:* The user clicks the "Station Zone Mapping" button on the workspace toolbar.
- **Actions / Postconditions:**
  - *Action:* A collapsible offcanvas drawer slides out containing a searchable list of all network stations and their assigned zone dimensions (`site-dimension-bind`).
  - *Postcondition:* The operator can reassign a station to a new zone and save the mapping without reloading or clearing the active fare matrix.

Acceptance Criteria

1. The drawer opens alongside the fare grid without reloading or clearing the current matrix state.
2. Displays all stations with: Station Code (`siteCode`), Station Name, and current Zone Assignment dropdown.
3. Includes a real-time search filter allowing quick lookup of stations by name or code.
4. Selecting a new zone from a station's dropdown highlights the row as modified.

5. Saving the drawer persists the station-to-zone binding changes and displays a confirmation badge.

Test Scenarios

- **Manual (Success):** Open the drawer, search for "Ravenswood", and change its zone from `Zone B` to `Zone A`. Click "Update Mappings". Verify the drawer reflects the new zone and shows a success toast.
- **Manual (Failure / Edge Case):** Attempt to submit an invalid station binding request via a corrupted payload (e.g., missing station code or invalid target zone identifier) or simulate a network failure when clicking "Update Mappings". Verify the server rejects the request with an HTTP 400 error banner ("Failed to update station mapping: invalid zone identifier"), and the UI displays an error alert without closing the drawer or losing entered changes.
- **Automated (Unit / Slice / Integration):** `MockMvc` POST request to `/api/topology/bind-station` with valid payload asserting HTTP 200; execute request with blank/invalid zone parameter asserting HTTP 400 Bad Request and verifying that database mappings remain unmutated.

Actionable Sub-tasks

- ☐ Implement Bootstrap 5 Offcanvas drawer component in `rate-grid.html`.
- ☐ Build station mapping table with instant JavaScript search/filter bar.
- ☐ Populate station rows with zone selection dropdowns.
- ☐ Add change-tracking listeners to flag modified station-zone pairs.
- ☐ Create backend controller endpoint handling station dimension bindings.

Story 2.6: Visual Draft Tracking, Diff Confirmation, & Platform Batch API Dispatch (Parent Story)

- **GitHub Issue Title:** `[Rate Grid] Visual Draft Tracking, Diff Confirmation, & Platform Batch API Dispatch`
- **Iteration:** `3`
- **Story Points:** `5` *(Split into sub-stories 2.6a and 2.6b)*
- **Story Description:** As an authenticated operator, I want all unsaved rate changes visually highlighted in draft state and summarized in a confirmation diff modal before committing, so that I can review old vs. new prices and batch-dispatch valid updates to Caracal's platform API.
- **Actors / Personas:** Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**

  - *Precondition:* The user has modified one or more cells (via inline editing, tier reassignment, or global tier adjustments).
  - *Trigger:* The user clicks "Save Rate Changes".
- **Actions / Postconditions:**
  - *Action:* The user reviews the diff modal detailing Pair/Route, Product/Tier, Old Price, New Price, and Scope. The user clicks "Confirm & Publish".
  - *Postcondition:* The Spring Boot backend validates the payload and dispatches a batch request to `/functions/od-matrix-set` with the operator's `X-Actor` header. On success, draft styling clears from the grid.

Acceptance Criteria

1. Any cell modified during the active session is styled with an amber draft highlight.
2. Each drafted cell provides an inline revert trigger (or Escape key) restoring its initial value.
3. A global "Discard Changes" button resets all drafted cells back to their loaded platform values.
4. The "Save Rate Changes" confirmation modal summarizes all pending modifications with explicit old vs. new values.
5. The backend validates the payload using Spring Bean Validation (`@DecimalMin("0.00")`, `@Digits(fraction=2)`) before invoking Caracal's API.
6. Outgoing mutating requests attach the operator's identity via `X-Actor: operator:<email>` and return a success banner showing the count of rules written.

Test Scenarios

- **Manual (Success):** Edit two fares, verify the floating action bar reads "2 Pending Changes". Click "Save Rate Changes", confirm the diff modal lists both modifications accurately, click "Confirm & Publish". Verify an alert confirms "2 price rules updated" and cells return to normal styling.
- **Manual (Failure / Edge Case):** Submit a payload containing a negative price (`-5.00`) via developer tools or corrupted state, or simulate platform API failure (HTTP 500). Assert the Spring backend blocks negative prices with HTTP 400, or displays an error alert ("Failed to persist changes to platform") while preserving draft highlights on the grid.
- **Automated (Unit / Slice / Integration):** WireMock integration test asserting `/functions/od-matrix-set` receives the expected payload and `X-Actor` header, returning HTTP 200; simulate HTTP 500 from upstream API asserting controller catches error and preserves client draft state.

Actionable Sub-tasks

- ☐ Implement client-side `DraftStateManager` tracking changed cells, initial values, and change counts.
- ☐ Add amber visual draft styling (`.table-warning`, `.cell-draft`) and single-cell revert buttons.
- ☐ Build "Save Changes" review modal dynamically rendering the diff table.
- ☐ Implement Spring Boot `RateBatchUpdateDto` with strict Bean Validation constraints (`@DecimalMin`, `@Digits`, `@NotNull`).
- ☐ Implement `CaracalApiClient` method calling `/functions/od-matrix-set` with active session `X-Actor` header.
- ☐ Write slice tests (`@WebMvcTest`) and WireMock integration tests covering batch dispatch and validation failure handling.

Story 2.6a: Visual Draft State Engine & Cell Revert Controls

- **GitHub Issue Title:** `[Rate Grid] Visual Draft State Engine & Cell Revert Controls`
- **Iteration:** `2`
- **Story Points:** `2`
- **Story Description:** As an authenticated operator, I want unsaved rate modifications visually highlighted on the grid with individual revert options, so that I can track pending edits and undo mistakes before saving.
- **Actors / Personas:** Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**
  - *Precondition:* An operator edits any price cell via inline editing or modal action.
  - *Trigger:* A rate value is modified or an availability state is toggled.
- **Actions / Postconditions:**
  - *Action:* The operator modifies cell values or clicks an inline revert button.
  - *Postcondition:* Modified cells receive an amber highlight (`.table-warning`), a toolbar badge displays the pending changes count, and the user can revert single cells or discard all edits.

Acceptance Criteria

1. Modified cells display an amber draft border/background class (`.table-warning`).

2. Hovering or clicking a drafted cell reveals a revert icon (or Escape key) to restore its initial value.
3. A floating status bar displays an active count of pending changes.
4. "Discard All" restores the entire grid back to loaded platform values.

Test Scenarios

- **Manual (Success):** Modify two cells. Verify both turn amber and the toolbar reads "2 Pending Changes". Click revert on one cell; verify it resets to its loaded platform value and the badge updates to "1 Pending Change".
- **Manual (Failure / Edge Case):** Attempt to trigger a revert action (or press Escape) on an unmodified cell that has no staged changes, or click "Discard All" when no edits are pending. Verify the cell styling remains normal, no draft events fire, and the pending changes counter remains at `0` without decrementing into negative numbers.
- **Automated (Unit / Slice / Integration):** JavaScript unit test verifying `DraftStateManager` tracks modifications, decrements on revert, and empties on discard; assert that calling `revertChange()` on an untracked cell returns `false` and does not decrement the counter below zero.

Actionable Sub-tasks

- ☐ Create `DraftStateManager.js` module tracking modified cell key-value pairs.
- ☐ Add CSS classes `.table-warning` and `.cell-draft` for modified table cells.
- ☐ Implement floating action bar showing pending changes count.
- ☐ Add individual revert button icon to cell templates.
- ☐ Implement "Discard All" button clearing the draft dictionary.

Story 2.6b: Transaction Review Diff Modal & Batch API Dispatch

- **GitHub Issue Title:** `[Rate Grid] Transaction Review Diff Modal & Batch API Dispatch`
- **Iteration:** `3`
- **Story Points:** `3`
- **Story Description:** As an authenticated operator, I want to review an itemized diff of all staged changes before batch-submitting them, so that I can verify accuracy and persist valid updates to the platform API under my audit identity.
- **Actors / Personas:** Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**

  - *Precondition:* One or more cells are in draft state.
  - *Trigger:* The operator clicks the "Save Rate Changes" button.
- **Actions / Postconditions:**
  - *Action:* A confirmation diff modal lists each change (Pair, Product/Tier, Old Rate, New Rate). Clicking "Confirm & Publish" initiates batch persistence.
  - *Postcondition:* The Spring Boot backend dispatches a batch request to `/functions/od-matrix-set` with the operator's `X-Actor` header and clears the draft state.

Acceptance Criteria

1. Diff modal displays an itemized list of all pending changes with Old vs. New rates.
2. Backend DTO validates currency fields with `@DecimalMin("0.00")` and `@Digits(fraction=2)`.
3. Outgoing platform request carries `X-Actor: operator:<email>`.
4. On HTTP 200 from the platform, draft state clears and a success banner displays the rules written count.
5. If the platform is unreachable or returns an error, draft state is retained and an error banner is displayed.

Test Scenarios

- **Manual (Success):** Click "Save Rate Changes" → review diff modal → confirm. Verify success toast appears ("X price rules updated") and amber styling clears from cells.
- **Manual (Failure / Edge Case):** In the diff review modal, click "Confirm & Publish" while the platform backend is offline or returns an error (HTTP 500 / 400). Verify an error banner displays ("Failed to persist changes to platform: upstream service error"), the modal remains open or preserves the staged list, and all amber draft borders remain on the grid so unsaved edits are not lost.
- **Automated (Unit / Slice / Integration):** WireMock integration test asserting `/functions/od-matrix-set` receives the expected payload and `X-Actor` header, returning HTTP 200; simulate HTTP 500 from `/functions/od-matrix-set` asserting the controller catches the failure, returns an error status to the client, and retains draft state.

Actionable Sub-tasks

☐ Build Bootstrap diff review modal displaying staged item changes.
☐ Implement Spring Boot `RateBatchUpdateDto` with Bean Validation annotations.
☐ Implement `CaracalApiClient.setOdMatrix()` passing `X-Actor` header.

☐ Connect frontend "Confirm & Publish" button to controller batch POST endpoint.
☐ Write WireMock tests covering success and HTTP 500 error handling.

Epic 3: Ticket Availability and Product Lifecycle Management

This document defines the formal user stories and technical tasks for Epic 3 of the NY Waterway Fare Configuration Tool. Each story adheres strictly to the CMPT 276 agile project specification, INVEST criteria, and standardized persona definitions.

Story 3.1: Terminal Ticket Availability UI Controls

- **GitHub Issue Title:** `[Availability] Terminal Ticket Availability UI Controls`
- **Iteration:** `2`
- **Story Points:** `3`
- **Story Description:** As an authenticated operator, I want interactive toggle switches for individual ticket classes across specific terminal rate tables, so that I can easily manage product availability and control offerings at each station.
- **Actors / Personas:** Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**
  - *Precondition:* An operator is logged into the application and navigates to the Availability view (`/availability`) with a terminal selected.
  - *Trigger:* The operator flips an availability toggle switch for a ticket product.
- **Actions / Postconditions:**
  - *Action:* The operator clicks a toggle switch corresponding to a ticket class.
  - *Postcondition:* The switch updates its visual state, changes the cell badge to `NOT_SOLD` or `SELLABLE`, and registers the modification in the client-side draft state manager.

Acceptance Criteria

1. Each ticket class per terminal displays an active/inactive toggle switch.

2. Disabling a ticket class instantly updates its visual status on the grid (e.g., displaying a muted `NOT_SOLD` badge).
3. Availability changes are captured as pending draft modifications before final commit.
4. Toggling an unconfigured route or invalid state displays an error notification and prevents toggling.

Test Scenarios

- **Manual (Success):** Select terminal `EDGEWTR`, locate `SENIOR_ONEWAY`, and flip its toggle to "Off". Verify the cell displays `NOT_SOLD` and the pending changes counter increments by 1.
- **Manual (Failure / Edge Case):** Attempt to toggle an availability switch while offline or when page state is stale. Verify an error alert appears ("Unable to stage change: stale state") and the toggle reverts to its original position.
- **Automated (Unit / Slice / Integration):** `@WebMvcTest` asserting `GET /availability?site=EDGEWTR` returns HTTP 200 with terminal products and toggle models; JavaScript unit test verifying toggle events update `DraftStateManager`.

Actionable Sub-tasks

- ☐ Build availability table view with Bootstrap toggle switches (`form-check-input`).
- ☐ Connect toggle change listeners to client-side `DraftStateManager`.
- ☐ Add visual badge rendering (`SELLABLE` vs `NOT_SOLD`).
- ☐ Implement controller endpoint `GET /availability` in Spring Boot.
- ☐ Write MockMvc slice tests for view rendering.

Story 3.2: Permanent Phase-Out Handling & Local Persistence

- **GitHub Issue Title:** `[Availability] Permanent Phase-Out Handling & Local Persistence`
- **Iteration:** `2`
- **Story Points:** `2`
- **Story Description:** As a Caracal administrator, I want permanent product phase-outs (such as network-wide Senior fare discontinuations) to persist across session reloads, so that discontinued tickets are permanently blocked without breaking underlying database schemas.
- **Actors / Personas:** Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**

  - *Precondition:* An operator or administrator is logged in with access to product lifecycle settings.
  - *Trigger:* The user flags a ticket product for permanent phase-out.
- **Actions / Postconditions:**
  - *Action:* The user confirms the permanent phase-out in a confirmation dialog.
  - *Postcondition:* The application updates the local PostgreSQL database to permanently disable the product class across all relevant regional rate tables, persisting across browser restarts.

Acceptance Criteria

1. Permanent phase-outs completely disable purchasing permissions (`NOT_SOLD`) for specified products across all terminals.
2. Phased-out states persist in PostgreSQL across session reloads and browser restarts.
3. Phased-out products cannot be re-enabled by standard `OPERATOR` users without `CLIENT_SERVICES` clearance.
4. The system prompts the user with an explicit confirmation dialog before applying a permanent phase-out.

Test Scenarios

- **Manual (Success):** Log in as `CLIENT_SERVICES`, flag Senior fares as permanently phased out, and refresh the browser. Verify the product remains disabled and labeled as phased out across all terminals.
- **Manual (Failure / Edge Case):** Log in as `OPERATOR` and attempt to re-enable a permanently phased-out ticket product. Verify action is blocked and a 403 Forbidden or permission alert displays.
- **Automated (Unit / Slice / Integration):** `@DataJpaTest` verifying phased-out entity flags persist in the local PostgreSQL repository and prevent unauthorized status mutations.

Actionable Sub-tasks

☐ Add `isPhasedOut` attribute to local product/availability schema.
☐ Build confirmation modal dialog for permanent phase-outs.
☐ Enforce `CLIENT_SERVICES` authorization checks for reactivating phased-out products.
☐ Write persistence tests using `@DataJpaTest`.

Story 3.3: Temporal Sales Window & Condition Builder Modal

- **GitHub Issue Title:** `[Availability] Temporal Sales Window & Condition Builder Modal`

- **Iteration:** `2`
- **Story Points:** `3`
- **Story Description:** As an authenticated operator, I want a dedicated modal to define and attach temporal availability conditions (such as monthly pass windows or weekday/weekend segregation) to specific ticket products, so that automated calendar rules and recurring sales windows are properly enforced.
- **Actors / Personas:** Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**
  - *Precondition:* An operator is on the availability matrix.
  - *Trigger:* The operator clicks the "Configure Condition" icon on a ticket product.
- **Actions / Postconditions:**
  - *Action:* The user selects a condition type (e.g., `DAY_OF_MONTH` for monthly passes, `WEEKDAY`, or `WEEKEND`) and sets date/time constraints.
  - *Postcondition:* The condition is validated and staged in draft state for that ticket assignment.

Acceptance Criteria

1. A dedicated modal interface allows configuration of date ranges and recurring monthly pass sales windows (e.g., 20th of the previous month to 10th of the current month).
2. Day-of-week filtering correctly isolates weekday commuter products from weekend fares.
3. Client-side and server-side validation ensures start dates/days are valid and prevents malformed inputs.
4. Modal shows current active conditions attached to the selected product.

Test Scenarios

- **Manual (Success):** Open condition modal for `FULL_MONTHLY`, select `DAY_OF_MONTH`, set start to 20 and end to 10, and click Save. Verify the modal closes and the product displays the monthly window badge.
- **Manual (Failure / Edge Case):** Enter an invalid day of month (e.g., start day 35 or negative values). Verify validation displays an error ("Day must be between 1 and 31") and blocks saving.
- **Automated (Unit / Slice / Integration):** JUnit service test verifying temporal condition validator correctly parses recurring monthly windows (20 to 10) and flags out-of-range days.

Actionable Sub-tasks

- ☐ Build Bootstrap condition builder modal in Thymeleaf template.
- ☐ Implement date/day-of-month form inputs with validation constraints.
- ☐ Add visual badge on availability grid indicating active temporal rules.
- ☐ Write JUnit unit tests for temporal window boundary parsing.

Story 3.4: Backend REST API Integration for Availability & When-Conditions

- **GitHub Issue Title:** `[Availability] Backend REST API Integration for Availability & When-Conditions`
- **Iteration:** `3`
- **Story Points:** `3`
- **Story Description:** As an authenticated operator, I want local availability toggles and temporal sales rules to synchronize with Caracal's backend via REST API endpoints, so that live ticket vending machines enforce the configured sales restrictions and remote rate assignments.
- **Actors / Personas:** Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**
  - *Precondition:* Availability edits or temporal conditions are in draft state.
  - *Trigger:* The operator clicks "Save Availability & Conditions".
- **Actions / Postconditions:**
  - *Action:* The Spring Boot backend serializes the staged changes and dispatches HTTP calls to `/functions/rate-assignment-update` and `/functions/when-condition-create`.
  - *Postcondition:* Caracal platform updates rate assignments, clears local draft state, and displays a success alert with the operator's `X-Actor` header recorded.

Acceptance Criteria

1. Spring WebClient successfully packages availability updates into requests targeting `/functions/rate-assignment-update`.
2. Temporal rules are transmitted to the `/functions/when-condition-create` endpoint using correct JSON schemas.
3. Authentication headers (`X-Actor: operator:<email>`) are properly attached to all outgoing API requests.

4. If Caracal's backend returns an error (4xx/5xx), draft state is retained and an error banner is displayed.

Test Scenarios

- **Manual (Success):** Toggle a ticket's availability, click Save, and verify a success banner appears. Check that `/functions/rate-table-get` reflects the new assignment status.
- **Manual (Failure / Edge Case):** Simulate Caracal API server downtime (HTTP 500). Verify an error alert displays ("Failed to synchronize availability with platform") and staged toggles remain in draft state.
- **Automated (Unit / Slice / Integration):** WireMock integration test asserting `POST /functions/rate-assignment-update` and `POST /functions/when-condition-create` receive correct JSON bodies and `X-Actor` headers, returning HTTP 200.

Actionable Sub-tasks

☐ Implement `CaracalApiClient.updateRateAssignment()` and `createWhenCondition()`.
☐ Add `X-Actor` header propagation from authenticated session.
☐ Build batch persistence service to coordinate assignment and condition updates.
☐ Write WireMock integration tests covering API success and failure paths.

Story 3.5: Automated Unit & Integration Testing for Epic 3

- **GitHub Issue Title:** `[Availability] Automated Unit & Integration Testing for Epic 3`
- **Iteration:** `3`
- **Story Points:** `2`
- **Story Description:** As a developer, I want automated unit and integration tests covering ticket availability toggles, temporal date parsers, and API payload serialization, so that regressions are prevented before merging code into main releases.
- **Actors / Personas:** Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**
  - *Precondition:* Availability modules and services are implemented.
  - *Trigger:* A developer runs local test suites or triggers GitHub Actions CI.
- **Actions / Postconditions:**

- *Action:* JUnit, Mockito, and MockMvc execute test routines across availability components.
- *Postcondition:* All tests pass, verifying draft state management, temporal condition evaluations, and mock endpoint responses.

Acceptance Criteria

1. Unit tests verify that toggling ticket availability updates the internal draft state model properly.
2. Test suites validate that the temporal condition parser correctly evaluates active sales windows.
3. Spring Boot integration tests mock external Caracal API endpoints to confirm payload accuracy.
4. CI test suite executes in under 60 seconds with 100% pass rate.

Test Scenarios

- **Manual (Success):** Run `./mvnw test` locally; verify all Epic 3 unit and slice tests execute with 0 failures and 0 errors.
- **Manual (Failure / Edge Case):** Introduce an intentional syntax error or failing assertion in `AvailabilityServiceTest`; run `./mvnw test` and verify Maven build fails with a descriptive test failure report.
- **Automated (Unit / Slice / Integration):** Maven CI build pipeline runs unit, slice (`@WebMvcTest`), and integration tests on pull requests.

Actionable Sub-tasks

☐ Write unit tests for `AvailabilityDraftManager`.
☐ Write unit tests for temporal date condition evaluator.
☐ Create MockMvc slice tests for availability controller endpoints.
☐ Add WireMock test suite for availability payload serialization.

Story 3.6: Documentation for Availability & Temporal Rules Management

- **GitHub Issue Title:** `[Availability] Documentation for Availability & Temporal Rules Management`
- **Iteration:** `3`
- **Story Points:** `1`
- **Story Description:** As an authenticated operator or developer, I want clear internal documentation detailing how to configure ticket availability toggles and temporal sales windows, so that team members and future maintainers understand the workflow and API mappings.

- **Actors / Personas:** Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**
  - *Precondition:* Epic 3 features are implemented.
  - *Trigger:* A developer or operator reviews repository documentation.
- **Actions / Postconditions:**
  - *Action:* Developer authors a markdown guide in the repository.
  - *Postcondition:* A complete documentation file outlines UI workflows, date constraint syntax, and backend API integration steps.

Acceptance Criteria

1. Documentation explains how operators interact with terminal availability toggles and condition modals.
2. API endpoints (`rate-assignment-update` and `when-condition-create`) are documented with expected JSON payloads.
3. Instructions for running Epic 3 unit and integration tests are clearly outlined for the engineering team.

Test Scenarios

- **Manual (Success):** Open `docs/availability-guide.md` and follow the steps to configure a monthly pass window; verify all documented UI controls and API schemas match the codebase.
- **Manual (Failure / Edge Case):** Verify that missing API fields or undocumented endpoints are flagged during peer review before merging the documentation PR.
- **Automated (Unit / Slice / Integration):** Markdown linter / CI check verifies document formatting, valid headers, and working internal file links.

Actionable Sub-tasks

☐ Author `docs/availability-guide.md` with UI workflow walkthrough.
☐ Document Caracal API request/response contracts for Epic 3.
☐ Add testing instructions and troubleshooting tips for developers.

Epic 4: Pre-Purchase Warnings and TVM Preview

When a rider picks a ticket that can't be used right now, the ticket machine (TVM) should show a short heads-up before they pay. The rider can still buy the ticket. The warning just makes sure they know when it will work.

Our client described three cases in our meetings so far:

- **Weekday-only routes:** On a Saturday, a rider picks Edgewater → Midtown and sees "This route doesn't run on weekends. Your ticket is valid Monday."
- **Rush-hour-only routes:** Some routes only run during weekday morning and evening rush hours, so riders outside those hours see a notice.
- **Advance-purchase promos:** A rider buys a December promo ticket on Oct 30 and sees "You can buy this now, but you can't use it until December 1."

These are **not** outage or maintenance messages. Every warning is shown in English and Spanish, like the client's current TVM screens. Caracal's platform API has no endpoints for warnings or route service hours, so this epic stores both in our own database. Epic 3 (Availability) decides when a ticket is sold, and Epic 4 decides what the rider is told.

Note: The character limits below (200 for messages, 20 for button labels) are our best guess. We'll confirm the character limit at a client sync.

Story 4.1: View Warning Messages

- **GitHub Issue Title:** `[Alerts] View Warning Messages`
- **Iteration:** `1`
- **Story Points:** `2`
- **Story Description:** As an operator, I want to see every saved warning for my tenant in one list, so that I know what riders are being told before I change anything.
- **Actors / Personas:** Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**
  - *Precondition:* The user is logged in and a tenant is selected (`NYW_SANDBOX` for now).
  - *Trigger:* The user clicks **Service Alerts & TVM Canvas** in the sidebar, which opens `/fares/alerts`.
- **Actions / Postconditions:**
  - *Action:* The app loads the warnings saved for the active tenant.
  - *Postcondition:* A table shows each warning's route, which tickets it applies to, when it shows, and the English and Spanish messages.

Acceptance Criteria

1. Only logged-in users can open the page.
2. Only warnings for the active tenant (`tenantCode`) are listed.

3. Each row shows the route (for example, Edgewater → Midtown), whether it applies to **All tickets** or one specific ticket (for example, Adult One-Way), when it shows (Weekends, Outside service hours, or Before a start date), and both messages.
4. Warnings with no Spanish message show a **Missing Spanish** badge.
5. If there are no warnings, the page says "No warnings yet" and shows a **New warning** button.

Test Scenarios

- **Manual (Success):** Log in as `operator`, open Service Alerts, and check that saved warnings are listed with their route, ticket scope, and messages.
- **Manual (Failure / Edge Case):** Log out and go to `/fares/alerts`. Check that you are sent to the login page. Then log in with no warnings saved and check that "No warnings yet" shows.
- **Automated (Unit / Slice / Integration):** `@WebMvcTest` with `MockMvc`: a logged-out `GET /fares/alerts` redirects to `/login`, and a logged-in request returns HTTP 200, the `alerts/index` view, and only warnings for `NYW_SANDBOX`.

Actionable Sub-tasks

- ☐ Create the `ServiceAlert` entity (with `tenantCode`) and `ServiceAlertRepository`.
- ☐ Add `GET /fares/alerts` to `AlertController`, filtered by the active tenant.
- ☐ Replace the placeholder in `templates/alerts/index.html` with the real table.
- ☐ Add the ticket scope column and the Missing Spanish badge.
- ☐ Write `MockMvc` tests for logged-in, logged-out, and tenant filtering.

Story 4.2: Create a Bilingual Warning

- **GitHub Issue Title:** `[Alerts] Create a Bilingual Warning`
- **Iteration:** `1`
- **Story Points:** `3`
- **Story Description:** As an operator, I want to write a warning for a route in English and Spanish and choose when it shows, so that riders know when their ticket will work before they pay.
- **Actors / Personas:** Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**
  - *Precondition:* The user is logged in and on the alerts page.

  - *Trigger:* The user clicks **New warning**, which opens `/fares/alerts/new`.

- **Actions / Postconditions:**
  - *Action:* The user picks an origin terminal and a destination, optionally picks one ticket (it defaults to **All tickets on route**), picks when the warning shows (**Weekends**, **Outside service hours**, or **Before a start date**, with a date), and writes the English and Spanish messages.
  - *Postcondition:* On **Save**, the warning is stored for the active tenant and the user goes back to the list with a "Warning saved" message. If something is wrong, the form comes back with the user's input kept and an error under each problem field.

Acceptance Criteria

1. Origin, destination, "show when", and English message are required. Ticket is optional and defaults to **All tickets on route**.
2. If "Before a start date" is picked, a start date is required.
3. Messages are 200 characters or fewer, and a live counter under each box shows how many are used (for example, "145/200").
4. The Spanish message is optional for now, but the list flags warnings without one.
5. Nothing is saved when validation fails.

Test Scenarios

- **Manual (Success):** Pick Edgewater → Midtown, leave ticket as **All tickets on route**, and pick **Weekends**. Enter "SERVICE NOTICE: No ferry service on this route on weekends. Your ticket is valid Monday." and "AVISO DE SERVICIO: No hay servicio de ferry en esta ruta los fines de semana. Su boleto es válido el lunes." Save and check that the warning appears with "Warning saved".
- **Manual (Success):** Pick a December promo ticket and **Before a start date** with Dec 1. Save and check that the list shows "Before Dec 1".
- **Manual (Failure / Edge Case):** Leave the English message blank and save. Check that "English message is required" shows and nothing is saved. Pick **Before a start date** with no date and check that "Start date is required" shows.
- **Manual (Failure / Edge Case):** Type past 200 characters. Check that the counter turns red and "Message must be 200 characters or fewer" shows on save.
- **Automated (Unit / Slice / Integration):** `@WebMvcTest` with `MockMvc`: a valid `POST /fares/alerts` redirects to the list and saves one warning, and a `POST` with a blank English message returns the form with a field error on `messageEn`.

Actionable Sub-tasks

- ☐ Create `AlertForm` with Bean Validation (`@NotBlank`, `@Size(max = 200)`).
- ☐ Add `GET /fares/alerts/new` and `POST /fares/alerts`.
- ☐ Create `AlertService` to save warnings with the active `tenantCode`.
- ☐ Build `templates/alerts/form.html` with terminal, destination, and optional ticket dropdowns.
- ☐ Write `alert-form.js` for the live character counters.
- ☐ Write `MockMvc` tests for valid and invalid input.

Story 4.3: Edit or Delete a Warning

- **GitHub Issue Title:** `[Alerts] Edit or Delete a Warning`
- **Iteration:** `2`
- **Story Points:** `2`
- **Story Description:** As an operator, I want to edit a warning, so that I can fix wording without starting over. As a Caracal administrator, I want to delete old warnings, so that the list stays clean.
- **Actors / Personas:** Authenticated Operator (`OPERATOR`) can edit. Caracal Administrator (`CLIENT_SERVICES`) can edit and delete.
- **Preconditions & Triggers:**
  - *Precondition:* At least one warning exists.
  - *Trigger:* The user clicks **Edit** or **Delete** on a warning.
- **Actions / Postconditions:**
  - *Action:* **Edit** opens the Story 4.2 form with the current values filled in. **Delete** asks for confirmation.
  - *Postcondition:* Saving updates that warning. Confirming a delete removes it from the database.

Acceptance Criteria

1. Every logged-in user sees **Edit**. Only `CLIENT_SERVICES` users see **Delete**.
2. The delete endpoint is restricted to `CLIENT_SERVICES` in `SecurityConfig` (the same way `/admin/**` is today), so hiding the button isn't the only protection.
3. Delete asks "Are you sure?" first.
4. Edits follow the same rules as Story 4.2.
5. Opening a warning that doesn't exist shows the "not found" page.

Test Scenarios

- **Manual (Success):** As `operator`, change a warning's message and save. Check that the list shows the new text. As `admin`, delete a warning and confirm. Check that it's gone.
- **Manual (Failure / Edge Case):** As `operator`, check that there's no Delete button, and that sending the delete request directly shows the 403 page. Open `/fares/alerts/9999/edit` and check that the not found page shows.
- **Automated (Unit / Slice / Integration):** `@WebMvcTest` with `MockMvc`: a delete as `OPERATOR` returns 403 and the warning still exists, and a delete as `CLIENT_SERVICES` removes it.

Actionable Sub-tasks

- ☐ Add `GET /fares/alerts/{id}/edit` and `POST /fares/alerts/{id}`.
- ☐ Add `POST /fares/alerts/{id}/delete` and a `hasRole("CLIENT_SERVICES")` rule for it in `SecurityConfig`.
- ☐ Hide the Delete button for operators in the template.
- ☐ Return the not found page for missing warnings.
- ☐ Write permission tests for both roles.

Story 4.4: Show When the Ticket Becomes Valid

- **GitHub Issue Title:** `[Alerts] Show When the Ticket Becomes Valid`
- **Iteration:** `2`
- **Story Points:** `3`
- **Story Description:** As a rider, I want the warning to tell me exactly when my ticket will work, so that I'm not confused about whether I can ride today.
- **Actors / Personas:** Riders see the result. Authenticated Operator (`OPERATOR`) and Caracal Administrator (`CLIENT_SERVICES`) set up the route service hours.
- **Preconditions & Triggers:**
  - *Precondition:* The route's service days and AM/PM rush-hour windows are saved in our local database. Caracal's backend has no endpoint for service hours, so we store them alongside the warnings.
  - *Trigger:* A rider picks a ticket that has a warning, at a time when the warning applies (or an operator previews that time in Story 4.6).
- **Actions / Postconditions:**

- *Action:* The app works out the next time the ticket is valid.
- *Postcondition:* The warning text includes it, for example "Valid Monday", "Next service 4:30 PM", or "Valid from December 1".

Acceptance Criteria

1. **Weekends:** On Saturday or Sunday, the warning names the next weekday.
2. **Outside service hours:** Outside the route's AM/PM service windows, the warning names the next service time.
3. **Before a start date:** Before the start date, the warning names that date.
4. When the ticket is usable right now, no warning shows.
5. All checks use the terminal's time zone (`America/New_York`), not the server's.
6. If a route has no service hours saved yet, the plain warning still shows, just without a time.

Test Scenarios

- **Manual (Success):** Preview Edgewater → Midtown on Saturday at 10 AM and check that the warning says "Valid Monday". Preview a rush-hour-only route on Wednesday at 1 PM and check that it gives the next service time. Preview the December promo ticket on Oct 30 and check that it says "Valid from December 1".
- **Manual (Failure / Edge Case):** Preview a route with no service hours saved and check that the plain warning shows without a time instead of crashing. Preview on Sunday at 11:59 PM New York time and check that the warning still shows, even though it's already Monday on the server's UTC clock.
- **Automated (Unit / Slice / Integration):** JUnit service test for `nextValidTime()` with Friday night, Saturday, Sunday 11:59 PM, Monday morning, Wednesday 1 PM, a date before a promo start, and a route with no hours saved.

Actionable Sub-tasks

☐ Create the `RouteSchedule` entity (service days, AM and PM windows) and `RouteScheduleRepository`.
☐ Agree with Athan (Epic 3) so availability rules and service hours don't overlap.
☐ Write `nextValidTime()` in `AlertService` using the terminal's time zone.
☐ Add the computed time to the warning text.
☐ Write JUnit tests for the edge cases.

Story 4.5: Clear Button Labels on the Warning Popup

- **GitHub Issue Title:** `[Alerts] Clear Button Labels on the Warning Popup`
- **Iteration:** `2`
- **Story Points:** `2`
- **Story Description:** As an operator, I want to replace the generic "Confirm" and "Dismiss" buttons with clearer labels, so that riders know what each button does.
- **Actors / Personas:** Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**
  - *Precondition:* The user is creating or editing a warning.
  - *Trigger:* The user types in the button label fields.
- **Actions / Postconditions:**
  - *Action:* The user sets a continue label and a go-back label, each in English and Spanish.
  - *Postcondition:* The labels are saved with the warning and shown on the TVM popup and in the preview.

Acceptance Criteria

1. Each warning has a continue label and a go-back label, in English and Spanish.
2. The form suggests a continue label based on the warning type: "Buy for [Day]" for weekend and service-hour warnings (for example, "Buy for Monday"), and "Buy now" for promo warnings.
3. Blank labels fall back to "Continue" / "Continuar" and "Go back" / "Regresar".
4. Labels are 20 characters or fewer so they fit on the TVM button.

Test Scenarios

- **Manual (Success):** Create a weekend warning and check that the continue label is pre-filled with "Buy for Monday". Change the go-back label to "Choose another" and check that the preview shows both.
- **Manual (Failure / Edge Case):** Clear both labels and check that the preview shows "Continue" and "Go back". Enter a 30-character label and check that "Label must be 20 characters or fewer" shows.
- **Automated (Unit / Slice / Integration):** JUnit test that blank labels return the default English and Spanish text, and that the suggested label matches the warning type.

Actionable Sub-tasks

- ☐ Create the `AlertAction` model for button labels.
- ☐ Add the label fields to `AlertForm` with `@Size(max = 20)`.
- ☐ Pre-fill the suggested continue label based on the warning type.
- ☐ Apply the default labels when blank.
- ☐ Write JUnit tests for defaults and suggestions.

Story 4.6: Live TVM Preview

- **GitHub Issue Title:** `[Alerts] Live TVM Preview`
- **Iteration:** `2`
- **Story Points:** `3`
- **Story Description:** As an operator, I want to see the warning popup as riders will see it, at any day and time I pick, so that I catch wording, translation, or timing problems before anything is published.
- **Actors / Personas:** Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`)
- **Preconditions & Triggers:**
  - *Precondition:* The user is on the create or edit warning form in a desktop browser.
  - *Trigger:* The user types in a field, switches language, or changes the preview date and time.
- **Actions / Postconditions:**
  - *Action:* A panel next to the form redraws a mock TVM popup.
  - *Postcondition:* The preview shows the current text and labels in the chosen language, and shows or hides the warning based on the picked time.

Acceptance Criteria

1. The preview box has a fixed width and aspect ratio that matches the physical TVM touchscreen, so it doesn't stretch with the browser window.
2. The preview updates on each keystroke without reloading the page.
3. An **EN / ES** toggle switches the language.
4. A date and time picker shows or hides the warning using the Story 4.4 rules.
5. Text that won't fit on the screen turns red.
6. If **ES** is picked and there's no Spanish message, the preview shows "Missing Spanish translation".

Test Scenarios

- **Manual (Success):** Type in the English message box and check that the preview updates with each letter. Click **ES** and check that the Spanish text and labels show.
- **Manual (Success):** Change the preview time to cover all three cases: Saturday 10 AM (weekend warning shows), Wednesday 1 PM on a rush-hour-only route (service hours warning shows), and Oct 30 for the December promo (promo warning shows). Then pick Wednesday 8 AM on a weekday route and check that no warning shows.
- **Manual (Failure / Edge Case):** Leave the Spanish message empty and click **ES**. Check that the missing translation warning shows. Resize the browser and check that the preview box keeps its shape.
- **Automated (Unit / Slice / Integration):** `@WebMvcTest` with `MockMvc`: `GET /fares/alerts/new` returns HTTP 200, includes the preview fragment, and loads `tvm-preview.js`.

Actionable Sub-tasks

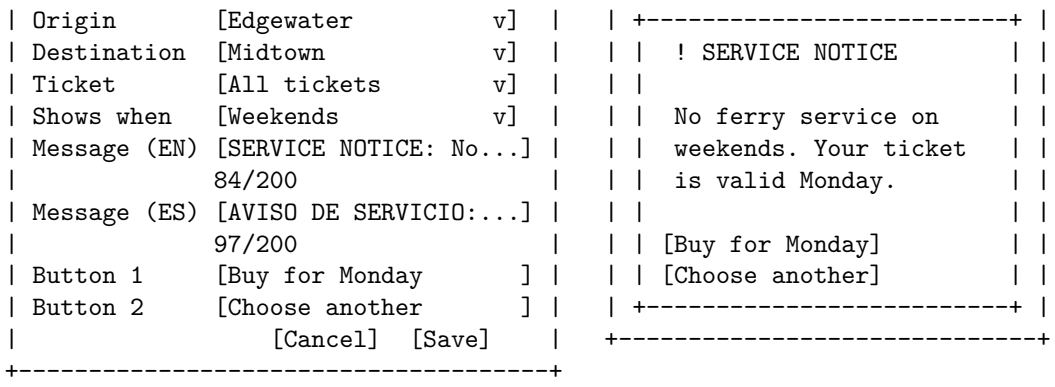
- ☐ Build `templates/alerts/tvm-preview.html` as a fragment with a fixed-size screen in `tvm-preview.css`.
- ☐ Write `tvm-preview.js` to update the preview on each `input` event.
- ☐ Add the EN / ES toggle and the date and time picker.
- ☐ Create `TvmPreviewModel` to pass warning data to the preview.
- ☐ Write the `MockMvc` test for the form page.

UI Mockups Warnings list (Stories 4.1 and 4.3)

```
+-------------------------------------------------------------------------------+
| Fare Config      |  Service Alerts & TVM Canvas              [+ New warning] |
|------------------|-----------------------------------------------------------|
| Fare Matrix      |  Route                Ticket       Shows when    Message  |
| Availability     |  ---------------------------------------------------------|
| > Service Alerts |  Edgewater → Midtown  All tickets  Weekends      SERVICE..|
| Publish          |                                           [Edit] [Delete*]|
|                  |  Port Imperial → P11  Dec Promo    Before Dec 1  You can..|
|                  |                       (Missing Spanish)   [Edit] [Delete*]|
|                  |                                                           |
|                  |  * Only Caracal administrators see Delete                 |
+-------------------------------------------------------------------------------+
```

Warning form with live preview (Stories 4.2, 4.5, and 4.6)

```
+--------------------------------------+   +------------------------------+
| New warning                          |   | TVM preview        [EN] [ES] |
|                                      |   | Preview at [Sat 10:00 AM  ]  |
```



Epic 5: Configuration Review, Validation, and Publishing

Story 5.1: View Pending Configuration Changes

- **GitHub Issue Title:** `[Epic 5] View Pending Configuration Changes`
- **Iteration:** `3`
- **Story Points:** `3`
- **Story Description:** As a **NY Waterway Operator**, I want to **review pending ticket configuration changes alongside the currently published configurations**, so that **I can identify and verify proposed modifications before they are published to ticket vending machines**.
- **Actors / Personas:** Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`).
- **Preconditions & Triggers:**
  - *Precondition:* The user is authenticated with either the `OPERATOR` or `CLIENT_SERVICES` role.
  - *Precondition:* The current `NYW_SANDBOX` configuration is accessible through the Caracal backend API.
  - *Precondition:* A reliable previously published configuration baseline is available through a supported publication endpoint or an application-maintained snapshot.
  - *Trigger:* The user navigates to the Configuration Review page (`GET /review`).
- **Actions / Postconditions:**
  - *Action:* The Spring Boot application retrieves the current sandbox configuration through the Java `CaracalApiClient`, using supported

endpoints such as `price-simulate` and `availability-simulate`.
- *Action:* The application retrieves the previously published configuration baseline through `CaracalApiClient` or an application-maintained snapshot.
- *Action:* The Java `ValidationService` compares the published and proposed configurations to identify changes in ticket prices and availability.
- *Action:* The application displays published and proposed values side by side, highlighting modified entries.
- *Postcondition:* The user can review pending configuration changes without modifying or publishing any data.
- *Postcondition:* If no differences exist, the application displays a message indicating that no changes are pending.

Acceptance Criteria

1. Both `OPERATOR` and `CLIENT_SERVICES` users can access the Configuration Review page. Unauthenticated users must log in before accessing it.
2. The application displays the current `NYW_SANDBOX` configuration alongside the previously published configuration.
3. Each identified change includes the affected origin terminal, destination route, ticket type, published value, and proposed value.
4. Changes in ticket prices and availability are visually highlighted, with unchanged entries clearly distinguishable.
5. If no differences exist, the application displays a message indicating that there are no pending changes.
6. If the Caracal API or published baseline cannot be retrieved, the application displays an appropriate error message rather than presenting incomplete comparison results.
7. Configuration retrieval and comparison are performed server-side through Java services. JavaScript is responsible only for frontend display and user interactions.
8. The comparison uses a reliable published baseline. The `publication-diff` endpoint is not used for direct draft-versus-published comparison because it compares the last two publications.
9. Viewing pending changes does not modify or publish configuration data.

UI Mockup Example of the Configuration Review page:

```
Configuration Review - NYW_SANDBOX

Origin    Destination   Ticket Type   Published   Proposed   Status
-------------------------------------------------------------------
EDGEWTR   MIDTOWN       Adult         $13.25      $14.00     Changed
EDGEWTR   MIDTOWN       Child         $7.00       $7.00      Unchanged
MIDTOWN   EDGEWTR       Senior        Sellable    Not Sold   Changed
```

`[Changed entries are visually highlighted]`

Test Scenarios

- **Manual (Success):** Log in as an `OPERATOR` and modify an adult ticket price in `NYW_SANDBOX` from $13.25 to $14.00. Open the Configuration Review page and verify that the published and proposed prices appear side by side, with the difference highlighted.
- **Manual (No Changes):** Open the Configuration Review page when the sandbox matches the published baseline. Verify that the application displays a message indicating that no changes are pending.
- **Manual (Failure / Edge Case):** Simulate an unavailable Caracal API or missing published baseline. Verify that the application displays an appropriate error message without crashing or showing misleading results.
- **Manual (Access Control):** Verify that both `OPERATOR` and `CLIENT_SERVICES` users can access the page. Attempt to access the page without authentication and verify that login is required.
- **Automated (Spring MVC Slice Test):** Use JUnit, `@WebMvcTest`, and `MockMvc` to verify that an authenticated `OPERATOR` or `CLIENT_SERVICES` request to `GET /review` returns HTTP 200 and includes the configuration comparison model attribute. Mock the comparison service with known published and proposed values. Verify that unauthenticated requests are rejected or redirected according to the Spring Security configuration.
- **Automated (JUnit Service Test):** Test `ValidationService` using known published and draft configurations. Verify that modified prices and availability statuses are identified correctly and matching values produce no pending differences.

Actionable Sub-tasks

- ☐ Create the Configuration Review page using HTML, CSS, and the application's frontend template system.
- ☐ Implement a Spring Boot controller endpoint for `GET /review`.
- ☐ Extend `CaracalApiClient` to retrieve current `NYW_SANDBOX` pricing and availability through `price-simulate` and `availability-simulate`.
- ☐ Retrieve the previously published configuration through a supported publication endpoint, such as `publication-payload-get` if its response contains the required data, or maintain a reliable application-side snapshot.
- ☐ Implement server-side Java comparison logic in `ValidationService` to identify differences between published and proposed configurations.
- ☐ Ensure comparisons use matching terminal, route, ticket type, and time contexts, including relevant weekday, weekend, and monthly availability conditions.
- ☐ Display published and proposed values side by side, with visual highlighting for modified entries.

- ☐ Enforce access for authenticated `OPERATOR` and `CLIENT_SERVICES` users through Spring Security.
- ☐ Handle missing baselines, empty comparisons, and backend retrieval failures.
- ☐ Implement automated JUnit and MockMvc tests for comparison results and access control.

Story 5.2: Validate Configuration Discrepancies

- **GitHub Issue Title:** `[Epic 5] Validate Configuration Discrepancies`
- **Iteration:** `3`
- **Story Points:** `3`
- **Story Description:** As a **NY Waterway Operator**, I want to **validate the sandbox ticket configurations against the provided reference dataset**, so that **I can identify incorrect settings and ensure all known discrepancies are resolved before publication**.
- **Actors / Personas:** Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`).
- **Preconditions & Triggers:**
  - *Precondition:* The user is authenticated with either the `OPERATOR` or `CLIENT_SERVICES` role.
  - *Precondition:* The `NYW_SANDBOX` configuration is accessible through the Caracal backend API.
  - *Precondition:* The reference dataset (`student-kit/data/nyw-sheet-2026-09-25.json`) is available to the Spring Boot application.
  - *Trigger:* The user navigates to the Configuration Validation page and clicks the "Run Validation" button.
- **Actions / Postconditions:**
  - *Action:* The Spring Boot application retrieves current sandbox ticket prices and availability through the Java `CaracalApiClient`, using supported endpoints such as `price-simulate` and `availability-simulate`.
  - *Action:* The Java `ValidationService.validateAgainstSheet()` compares the sandbox configuration against the expected values in `nyw-sheet-2026-09-25.json`.
  - *Action:* The service identifies discrepancies in ticket prices, ticket availability, and monthly pass sale windows.
  - *Action:* The application displays the detected discrepancies, their expected and actual values, and the total number of remaining issues.

- *Postcondition:* Users can identify incorrect configurations and use the appropriate editing features to correct them before publication.
- *Postcondition:* Users can rerun validation after configuration changes to obtain updated results.
- *Postcondition:* When all known discrepancies are resolved, the application displays a message indicating that no differences remain.

Acceptance Criteria

1. Both `OPERATOR` and `CLIENT_SERVICES` users can access and initiate configuration validation. Unauthenticated users must log in before accessing the page.
2. The application retrieves the current `NYW_SANDBOX` configuration through the Java `CaracalApiClient`, without making direct browser requests to the Caracal API.
3. The Java `ValidationService.validateAgainstSheet()` compares the sandbox configuration against the expected values in `student-kit/data/nyw-sheet-2026-09-25.json`.
4. Validation identifies discrepancies in ticket prices, ticket availability, and monthly pass sale windows.
5. Each discrepancy displays its category, affected origin terminal or destination route, ticket type, expected value, and actual value, where applicable.
6. The application displays the total number of remaining discrepancies and a breakdown by discrepancy category.
7. The initial seeded `NYW_SANDBOX` configuration produces the 50 known discrepancies: 30 incorrect prices, 15 tickets incorrectly available, 4 missing tickets, and 1 incorrect monthly sale window.
8. Users can rerun validation after configuration changes, and the displayed results reflect the latest sandbox configuration.
9. When all known discrepancies are resolved, the application displays a message indicating that no differences remain.
10. If the reference dataset is missing, malformed, or cannot be loaded, or if the Caracal API is unavailable, the application displays an appropriate error message rather than incomplete validation results.
11. Validation is read-only and does not modify or publish configuration data.
12. Configuration comparison and validation logic are implemented server-side in Java. JavaScript is used only for frontend interactions and displaying results.

Test Scenarios

- **Manual (Success):** Log in as an `OPERATOR`, open the Configuration Validation page, and click "Run Validation" against a freshly seeded `NYW_SANDBOX` configuration. Verify that the application reports 50 discrepancies: 30 incorrect prices, 15 tickets incorrectly available, 4 missing tickets, and 1 incorrect monthly sale window.

- **Manual (Partially Resolved):** Correct one or more known discrepancies using the configuration editing features, then rerun validation. Verify that the updated results reflect the corrected values and that the remaining discrepancy count decreases appropriately.
- **Manual (No Discrepancies):** Run validation after all known discrepancies have been corrected. Verify that the application displays a message indicating that no differences remain.
- **Manual (Failure / Edge Case):** Simulate an unavailable Caracal API, missing reference JSON file, or malformed reference data. Verify that the application displays an appropriate error message without crashing or presenting incomplete results as valid.
- **Manual (Access Control):** Verify that both `OPERATOR` and `CLIENT_SERVICES` users can initiate validation. Attempt to access the page without authentication and verify that login is required.
- **Automated (JUnit Service Test):** Use JUnit to test `ValidationService.validateAgainstSheet()` with a known reference dataset and mocked Caracal responses. Verify that an incorrect ticket price and an incorrectly available ticket are reported with the correct category, expected value, and actual value. Verify that matching values produce no discrepancies.
- **Automated (Spring MVC Slice Test):** Use `@WebMvcTest` and `MockMvc` to verify that authenticated `OPERATOR` and `CLIENT_SERVICES` users can access the validation page and initiate validation successfully. Verify that unauthenticated requests are rejected or redirected according to the Spring Security configuration.

Actionable Sub-tasks

- ☐ Create the Configuration Validation page using HTML, CSS, and the application's frontend template system.
- ☐ Implement Spring Boot controller endpoints to display the validation page and initiate validation.
- ☐ Extend `CaracalApiClient` to retrieve sandbox ticket prices and availability using `price-simulate` and `availability-simulate`.
- ☐ Load and parse `student-kit/data/nyw-sheet-2026-09-25.json` in the Spring Boot application.
- ☐ Implement `ValidationService.validateAgainstSheet()` in Java to compare the sandbox configuration against the reference dataset.
- ☐ Use `student-kit/tools/check-against-sheet.mjs` as a reference for the expected comparison behavior, including weekday/weekend ticket handling and monthly sale windows.
- ☐ Implement detection of pricing, availability, missing-ticket, and monthly sale-window discrepancies.

- ☐ Display detected discrepancies with their category, affected terminal or route, ticket type, expected value, and actual value.
- ☐ Display the total number of remaining discrepancies and counts for each category.
- ☐ Add a "Run Validation" button that allows users to rerun validation after configuration changes.
- ☐ Enforce access for authenticated `OPERATOR` and `CLIENT_SERVICES` users through Spring Security.
- ☐ Handle API failures, missing or malformed reference data, and configurations with no discrepancies.
- ☐ Implement automated JUnit tests for `ValidationService` and MockMvc tests for controller behavior and access control.
- ☐ Verify the initial 50 discrepancies and confirm that validation reports zero differences after all known issues are corrected.

Story 5.3: Publish Verified Configurations

- **GitHub Issue Title:** `[Epic 5] Publish Verified Configurations`
- **Iteration:** `3`
- **Story Points:** `2`
- **Story Description:** As a **Caracal Administrator**, I want to **publish reviewed and validated NY Waterway ticket configurations**, so that **updated ticket prices and availability settings can be safely released for use by ticket vending machines**.
- **Actors / Personas:** Caracal Administrator (`CLIENT_SERVICES`), Authenticated Operator (`OPERATOR`) with view-only access.
- **Preconditions & Triggers:**
  - *Precondition:* The user is authenticated with the `CLIENT_SERVICES` role to publish configurations.
  - *Precondition:* The current `NYW_SANDBOX` configuration is accessible through the Caracal backend API.
  - *Precondition:* The proposed configuration changes have been reviewed through the Configuration Review page from Story 5.1.
  - *Precondition:* Configuration validation from Story 5.2 has completed successfully with zero remaining discrepancies, and the validated configuration has not changed since validation.
  - *Trigger:* An authorized administrator clicks the "Publish Configuration" button and confirms the publication.
- **Actions / Postconditions:**

- *Action:* The Spring Boot application verifies that the current configuration has been reviewed and successfully validated before allowing publication.
- *Action:* The application presents a confirmation prompt describing the configuration being published.
- *Action:* After confirmation, the Spring Boot backend sends a POST request to `/functions/catalog-publish` through the Java `CaracalApiClient`, with `tenantCode` set to `NYW_SANDBOX` and the required `X-Actor` header.
- *Action:* The application retrieves publication records using `/functions/publication-list` to verify the new publication.
- *Postcondition:* If publication succeeds and the new record is verified, the application displays a success message with the available publication details.
- *Postcondition:* If publication fails or its result cannot be verified, the application displays an appropriate error or unverified-status message without falsely reporting a successful release.
- *Postcondition:* The published configuration is recorded in the backend publication history.

Acceptance Criteria

1. Only authenticated users with the `CLIENT_SERVICES` role can initiate publication. `OPERATOR` users can view the publication interface but cannot publish.
2. The Publish Configuration button is disabled for `OPERATOR` users. Direct publication requests from an `OPERATOR` must return HTTP 403.
3. Publication is blocked unless the proposed configuration has been reviewed and validation has completed successfully with zero remaining discrepancies.
4. The application ensures that validation still applies to the current sandbox configuration before publishing, preventing publication of changes introduced after validation.
5. The application requires explicit confirmation before submitting a publication request. Cancelling the confirmation does not trigger publication.
6. Upon confirmation, the Spring Boot backend sends a POST request to `/functions/catalog-publish` through `CaracalApiClient` with the request body `{"tenantCode":"NYW_SANDBOX"}` and an appropriate `X-Actor` header.
7. The application checks `/functions/publication-list?tenant=NYW_SANDBOX` to verify that a new publication record exists.
8. A publication is reported as successful only when the backend confirms the operation and the expected publication record is verified.
9. If the publication request fails, the application displays an appropriate error message without reporting success.
10. The application prevents duplicate publication requests while a publication

is in progress.

11. Publishing authorization, review requirements, and validation checks are enforced server-side through Spring Boot. JavaScript handles the confirmation prompt and frontend button state.
12. Publication requests are associated with the initiating actor through the `X-Actor` header, supporting backend traceability.

Test Scenarios

- **Manual (Success):** Log in as a `CLIENT_SERVICES` administrator, review the proposed configuration, and complete validation with zero remaining discrepancies. Click "Publish Configuration", confirm the action, and verify that publication succeeds and a new record appears in the publication history.
- **Manual (Validation Failure):** Attempt to publish while unresolved configuration discrepancies remain. Verify that the application blocks publication and displays a message requiring successful validation.
- **Manual (Configuration Changed):** Complete validation successfully, then modify the sandbox configuration before publishing. Verify that publication is blocked until the current configuration is validated again.
- **Manual (Cancellation):** Click "Publish Configuration" but cancel the confirmation prompt. Verify that no publication request is submitted.
- **Manual (Failure / Edge Case):** Simulate an unavailable Caracal API, a failed publication request, or an inability to verify the publication record. Verify that the application displays an appropriate error or unverified-status message without falsely reporting success.
- **Manual (Duplicate Request):** Click the Publish Configuration button multiple times while publication is processing. Verify that only one request is submitted.
- **Manual (Access Control):** Log in as an `OPERATOR` and verify that the Publish Configuration button is disabled. Attempt to submit a direct publication request and verify HTTP 403. Verify that unauthenticated users cannot publish.
- **Automated (WireMock Integration Test):** Use JUnit and WireMock to mock the Caracal backend. Verify that an authorized publication request sends `POST /functions/catalog-publish` with `{"tenantCode":"NYW_SANDBOX"}` and the required `X-Actor` header. Mock a successful publication response and a matching record from `GET /functions/publication-list?tenant=NYW_SANDBOX`, then verify that the application's publication endpoint returns HTTP 200.
- **Automated (Spring Security / MockMvc Test):** Use `MockMvc` to verify that `CLIENT_SERVICES` can initiate publication after review and

validation, while `OPERATOR` receives HTTP 403. Verify that failed validation prevents the publishing service from being called.

Actionable Sub-tasks

- ☐ Create a publication interface with a Publish Configuration button using HTML, CSS, and the application's frontend template system.
- ☐ Implement a Spring Boot controller endpoint for initiating publication.
- ☐ Integrate review and validation results from Stories 5.1 and 5.2 into the publication workflow.
- ☐ Implement server-side checks to ensure that the current configuration has been reviewed and validated with zero remaining discrepancies.
- ☐ Ensure publication is blocked if the sandbox configuration changes after successful validation, requiring validation of the updated configuration.
- ☐ Add a JavaScript confirmation prompt before submitting a publication request.
- ☐ Extend `CaracalApiClient` to send `POST /functions/catalog-publish` with the correct tenant code and `X-Actor` header.
- ☐ Retrieve publication records using `GET /functions/publication-list?tenant=NYW_SANDBOX` to verify the publication result.
- ☐ Display publication success, failure, and unverified-status messages appropriately.
- ☐ Disable the Publish Configuration button while a request is processing and prevent duplicate submissions server-side.
- ☐ Enforce `CLIENT_SERVICES` publishing authorization through Spring Security and return HTTP 403 for unauthorized `OPERATOR` requests.
- ☐ Implement automated JUnit, WireMock, and MockMvc tests for publication requests, validation requirements, and access control.

Story 5.4: View Configuration Change and Publication History

- **GitHub Issue Title:** `[Epic 5] View Configuration Change and Publication History`
- **Iteration:** `3`
- **Story Points:** `2`
- **Story Description:** As a **NY Waterway Operator**, I want to **view the history of ticket configuration changes and publications**, so that **I can track who made changes, when they were made, and maintain accountability for configuration updates**.
- **Actors / Personas:** Authenticated Operator (`OPERATOR`), Caracal Administrator (`CLIENT_SERVICES`).
- **Preconditions & Triggers:**

  - *Precondition:* The user is authenticated with either the `OPERATOR` or `CLIENT_SERVICES` role.
  - *Precondition:* The `NYW_SANDBOX` environment is accessible through the Caracal backend API.
  - *Trigger:* The user navigates to the Configuration History page (`GET /history`).

- **Actions / Postconditions:**
  - *Action:* The Spring Boot application retrieves configuration change records using `/functions/audit-log-get` through the Java `CaracalApiClient`.
  - *Action:* The application retrieves publication records using `/functions/publication-list` through `CaracalApiClient`.
  - *Action:* The application displays configuration activity, including the operation performed, responsible actor, timestamp, and additional details where available.
  - *Action:* The application displays publication history using the information returned by the backend.
  - *Postcondition:* Users can review previous configuration changes and publication records without modifying existing configuration data.
  - *Postcondition:* If no history records are available, the application displays an appropriate empty-state message.

Acceptance Criteria

1. Both `OPERATOR` and `CLIENT_SERVICES` users can access the Configuration History page. Unauthenticated users must log in before accessing it.
2. The application retrieves configuration change records for `NYW_SANDBOX` through `/functions/audit-log-get` using the Java `CaracalApiClient`.
3. Each audit record displays the operation performed, responsible actor, and timestamp, where those fields are provided by the backend.
4. The application displays additional configuration change details when available in the audit record.
5. The application retrieves and displays publication records through `/functions/publication-list?tenant=NYW_SANDBOX`.
6. Publication records display available information such as publication identifier, timestamp, and associated actor when provided by the backend.
7. Configuration changes and publications are clearly distinguishable in the history interface.
8. If no audit or publication records exist, the application displays a message indicating that no history is available.
9. If the Caracal API is unavailable or a history request fails, the application displays an appropriate error message rather than misleading or incomplete results.
10. Configuration history retrieval is performed server-side through Java services. JavaScript is responsible only for frontend display and interactions.

11. Viewing configuration history is read-only and does not modify or publish configuration data.

Test Scenarios

- **Manual (Success):** Log in as an `OPERATOR` or `CLIENT_SERVICES` user after a ticket price has been modified in `NYW_SANDBOX`. Open the Configuration History page and verify that the corresponding audit record displays the operation, responsible actor, timestamp, and available change details.
- **Manual (Publication History):** Publish a verified configuration through `catalog-publish`. Open the Configuration History page and verify that the new publication appears in the publication history with the available publication details.
- **Manual (No History):** Open the Configuration History page when the backend returns empty audit and publication record collections. Verify that the application displays a message indicating that no history is available.
- **Manual (Failure / Edge Case):** Simulate an unavailable Caracal API or a failed history retrieval request. Verify that the application displays an appropriate error message without crashing or presenting incomplete results as complete history.
- **Manual (Access Control):** Verify that both `OPERATOR` and `CLIENT_SERVICES` users can access the Configuration History page. Attempt to access the page without authentication and verify that login is required.
- **Automated (Spring MVC Slice Test):** Use JUnit, `@WebMvcTest`, and `MockMvc` to verify that authenticated `OPERATOR` and `CLIENT_SERVICES` requests to `GET /history` return HTTP 200 and include the expected audit and publication history model attributes. Mock the Java history service to return sample records originating from `audit-log-get` and `publication-list`. Verify that unauthenticated requests are redirected or rejected according to the Spring Security configuration.
- **Automated (JUnit Service Test):** Test the Java history service using mocked `CaracalApiClient` responses. Verify that audit and publication records are mapped correctly, empty responses produce an empty history state, and backend failures are handled appropriately.

Actionable Sub-tasks

- ☐ Create the Configuration History page using HTML, CSS, and the application's frontend template system.
- ☐ Implement a Spring Boot controller endpoint for `GET /history`.
- ☐ Extend `CaracalApiClient` to retrieve configuration change records through `/functions/audit-log-get`.

- ☐ Extend `CaracalApiClient` to retrieve publication records through `/functions/publication-list?tenant=NYW_SANDBOX`.
- ☐ Implement a Java history service to retrieve and organize audit and publication records for display.
- ☐ Display audit records with operation, responsible actor, timestamp, and available configuration change details.
- ☐ Display publication records with available publication identifiers, timestamps, and actor information.
- ☐ Clearly distinguish configuration changes from publication events in the history interface.
- ☐ Handle empty audit and publication records with an appropriate message.
- ☐ Handle Caracal API failures and missing optional record fields without crashing or displaying misleading results.
- ☐ Enforce access for authenticated `OPERATOR` and `CLIENT_SERVICES` users through Spring Security.
- ☐ Implement automated JUnit and MockMvc tests for history retrieval, empty states, backend failures, and access control.

Process and Workflow Documentation — Iteration 1

Development Methodology: Agile & Incremental Delivery

Our team employs an **Agile development methodology** structured around two-week iterative cycles. In accordance with modern software engineering principles, we prioritize delivering functional, demonstrable vertical slices of software over isolated, purely theoretical prototypes.

Client Syncs and Stakeholder Feedback To ensure continuous alignment with customer requirements and system constraints:

- **Weekly Client Meetings:** We hold weekly sync meetings with our external client, every Friday at 11:00 AM via video call.
- **Meeting Objectives:** These sessions serve to demo working software increments, validate UI/UX interactions (such as the Fare Rate Grid and Alert Designer workflows), refine upcoming iteration scope, and ensure that our Spring Boot application integrates seamlessly with Caracal's platform specifications and data models.
- **Meeting Records:** Detailed notes, action items, and design decisions are logged following each session to preserve historical context and drive sprint planning.

Project Management & GitHub Projects Kanban Board

We utilize **GitHub Projects** as our central project management hub to track every deliverable from ideation to production. The project board operates on

a customized **Kanban** framework to ensure total visibility into team velocity, active blockers, and task status.

Board Columns and Lifecycle Stages Every piece of engineering work is tracked through five progressive columns:

1. **Backlog:** Prospective features, long-term technical debt, and future user stories scheduled for Iterations 2 and 3.
2. **Todo:** Approved and estimated user stories prioritized specifically for the active iteration sprint.
3. **In Progress:** Issues currently under active local development by assigned team members.
4. **In Review:** Completed implementations awaiting peer review via Pull Request (PR) or staging validation on the Render deployment.
5. **Done:** Fully merged, tested, and accepted features that satisfy all defined acceptance criteria and pass CI/CD smoke tests.

Issue Metadata & Custom Fields To satisfy CMPT 276 traceability standards and facilitate accurate velocity measurement, each GitHub issue is enriched with standardized metadata:

- **Issue Labels:** Categorizes the nature of the task (e.g., `story` for functional user stories, `enhancement` for feature extensions, `bug` for defect resolution, and `documentation` for milestone deliverables).
- **Epic Custom Field:** Maps each issue back to one of the five core project epics:
  - *Epic 1:* Authentication, RBAC & User Management
  - *Epic 2:* Interactive Fare Rate Grid
  - *Epic 3:* Product Availability & Temporal Rules
  - *Epic 4:* TVM Alert Designer & Canvas Simulator
  - *Epic 5:* Staging, Discrepancy Auditing & Catalog Publication
- **Story Points Field:** Numerical complexity estimates calculated during sprint planning to evaluate sprint velocity.
- **Iteration Field:** Specifies the release milestone target (e.g., `Iteration 1`, `Iteration 2`, `Iteration 3`).

Version Control, Branching & Pull Request Workflow

We enforce a strict Git workflow to preserve repository stability, guarantee code quality, and maintain individual contribution traceability for grading purposes.

Feature Branch Strategy

- Developers never commit directly to the `main` branch.
- For every GitHub issue, the assigned developer checks out a dedicated feature branch using standardized naming conventions:

  - `feature/<epic-id>-<short-description>` (e.g., `feature/epic2-rate-grid`)
  - `docs/<topic>` (e.g., `docs/iteration1-user-stories`)
- This isolation prevents merge collisions, promotes focused code changes, and allows concurrent development across all five team members.

Pull Requests and Review Standards

- **Issue Linking:** When an implementation is complete, the author opens a Pull Request (PR) targeting `main`. The PR description must explicitly reference the associated issue using linking keywords:
  - Use `Refs #<issue-id>` for incremental or collaborative updates.
  - Use `Closes #<issue-id>` or `Fixes #<issue-id>` on final integration PRs to automatically transition the Kanban card to **Done**.
- **Peer Code Review:** Branch protection rules require at least one approving peer review before merging. Reviewers evaluate:
  - Code clarity, architectural decoupling, and object-oriented design standards.
  - Server-side and client-side input validation.
  - Accompanying unit/integration tests (`@WebMvcTest`, `@DataJpaTest`).
- **Continuous Deployment:** Merging a reviewed PR into `main` automatically triggers Render's build pipeline, updating the live staging environment.
- **Automated CI Verification (GitHub Actions):** To prevent regressions and ensure build stability before merging into `main`, an automated GitHub Actions continuous integration (CI) workflow triggers on every pull request and new commit. The pipeline sets up the Java environment and executes the complete test suite using Maven (`./mvnw clean test`). Merging is blocked until all unit and integration tests pass successfully alongside mandatory peer review approval.

Team Communication & Iteration 1 Retrospective

Communication Infrastructure To support asynchronous collaboration and course accountability, we maintain two primary communication channels:

- **Private Team Discord Server/Group Chat:** Used for real-time day-to-day coordination, impromptu voice huddles, screen-sharing technical debugging sessions, and sharing fast code snippets.
- **CMPT 276 Course Discord (Private Group Thread):** Used for formal announcements, course administrative syncs, and providing visibility to the course instructor (Bobby Chan) and Teaching Assistants.

Retrospective: Communication Challenges & Process Improvement During Iteration 1, our primary team obstacle was **asynchronous communication friction**:

- **The Challenge:** While each developer successfully built their assigned functional components, reliance on sporadic text chat led to communication gaps regarding active branch states, backend model assumptions, and frontend UI expectations. This occasionally resulted in redundant questions, delayed pull request reviews, and uncertainty about current blocker statuses across teammates.
- **The Process Improvement (Daily Scrums):** As part of our Iteration 1 retrospective, we are instituting **daily weekday scrums** (10–15 minutes, held virtually or via a structured daily standup prompt in Discord).
- **Standup Format:** Each member answers three standard questions:
  1. *What did I complete yesterday?*
  2. *What am I working on today?*
  3. *Are there any blockers impeding my progress?*
- **Target Outcome:** Implementing daily scrums will ensure the entire team stays synchronized, provides immediate peer assistance when blockers arise, and creates an unambiguous understanding of project momentum heading into Iteration 2.

Work Distribution & Ownership Matrix To maintain equal workload distribution and clear Git traceability across our five team members, primary ownership for Iteration 1 deliverables was allocated as follows:

Team Member	Primary Role & Epic Ownership	Iteration 1 Focus & Core Deliverables
Harendeep Singh	**Epic 1 Lead** (Authentication & RBAC)	Spring Security filter chain, BCrypt hashing, login/logout session handling, and user management views.
Oliver Tadaniewicz	**Epic 2 Lead** (Fare Matrix) & Client Liaison	Interactive Fare Matrix prototype, 2D grid layout, inline editing draft state, and meeting agendas.
Tejas Dutt	**Epic 3 Lead** (Availability Engine) & DB	PostgreSQL schema definitions, `AppUser` JPA entities, and database seeding scripts (`CommandLineRunner`).
Lakshya Joshi	**Epic 4 Lead** (Alerts & TVM Canvas)	Alert Designer layout, TVM preview canvas styling, and login error alert templates.

Team Member	Primary Role & Epic Ownership	Iteration 1 Focus & Core Deliverables
Athan Liu	**Epic 5 Lead** (Publishing/Review) & DevOps	Render hosting configuration, PostgreSQL connection pooling, GitHub Actions CI pipeline, and build automation.

Collaborative Engineering & Pair Programming In addition to asynchronous feature branching, our team utilized synchronous **pair programming sessions** via Discord voice channels with screen-sharing:

- **Security & Database Integration:** The Epic 1 and Epic 3 leads pair-programmed the Spring Security integration with PostgreSQL, using a driver-navigator approach to verify password hashing and role-based session authorities.
- **Matrix Layout & State Scaffolding:** Frontend and client leads paired on structuring the dynamic HTML table iteration and Thymeleaf bindings for origin-destination routes.
- **Peer Code Reviews:** As documented in our branching standards, every feature branch required peer review and verification before merging into `main`.